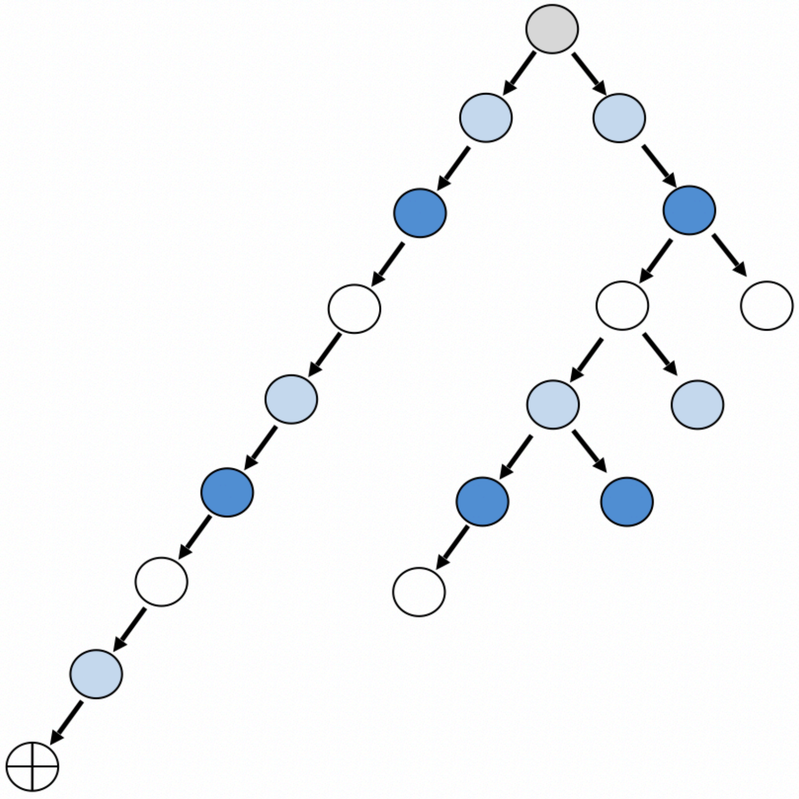
ROB-UY 3303

Robot Motion & Planning

Lecture 09A - Graph Search



Agenda

1. The Graph
2. Graph Search
3. Breadth First Search
4. Depth First Search

Agenda

1. **The Graph**
2. Graph Search
3. Breadth First Search
4. Depth First Search

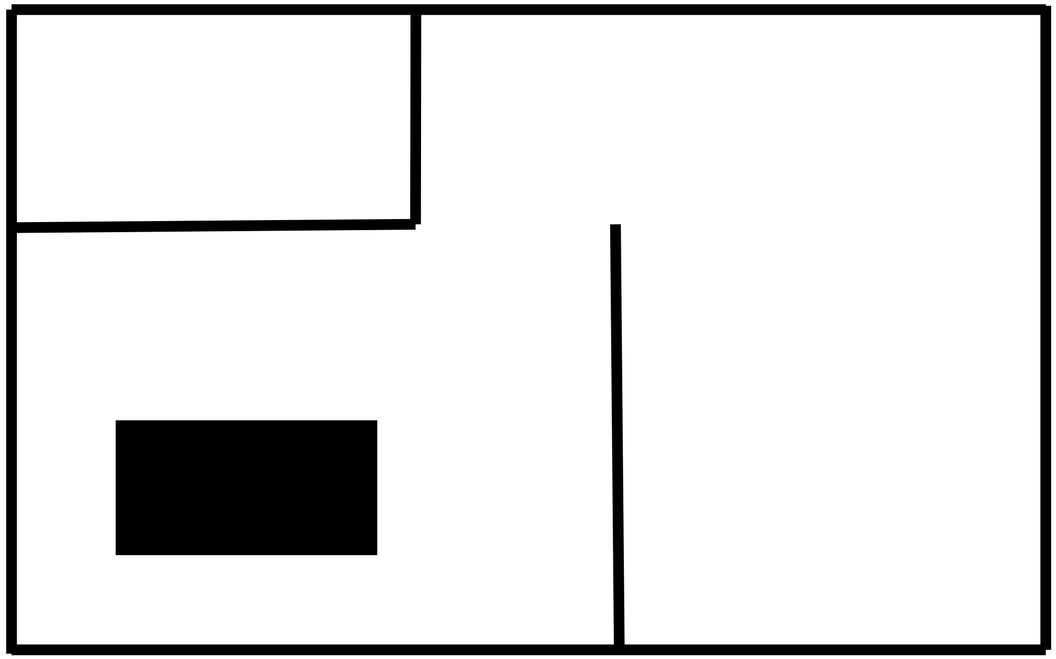
- **Graph Generation**
  - Each application and algorithm can have a different graph representation



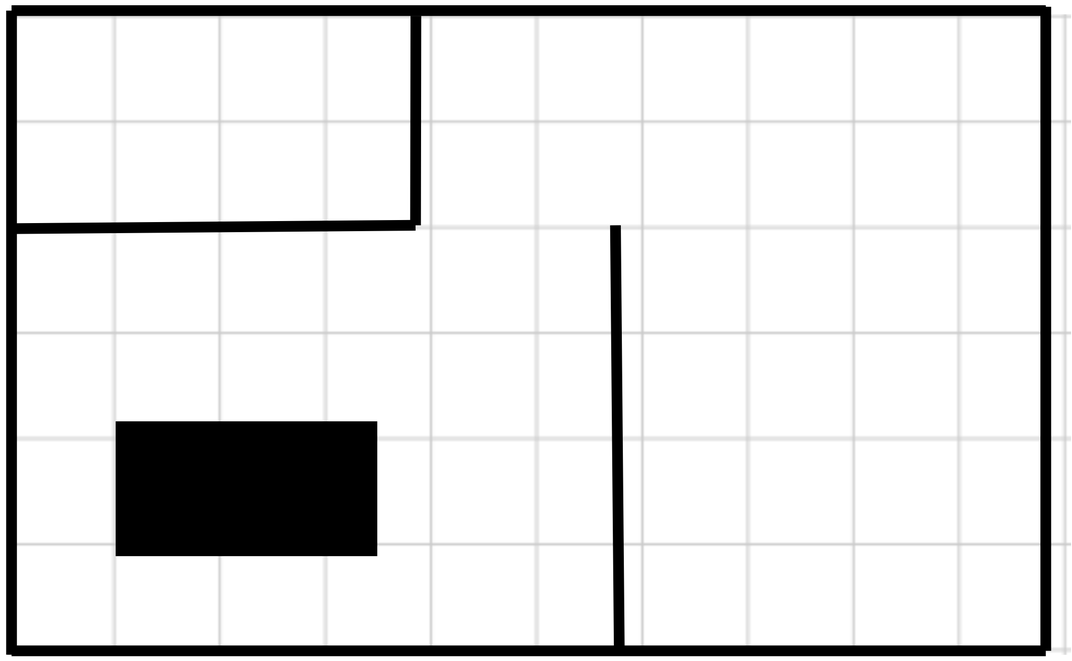
- **Graph Generation**
  - Often there are boundaries, and static objects to avoid.



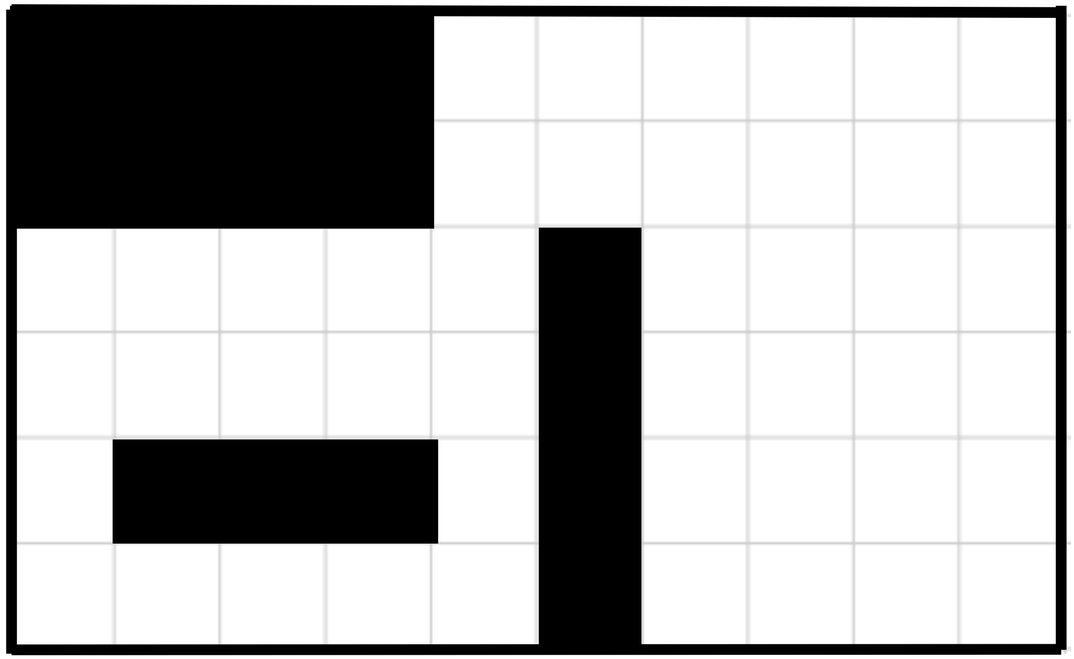
- **Graph Generation**
  - Often there are boundaries, and static objects to avoid.



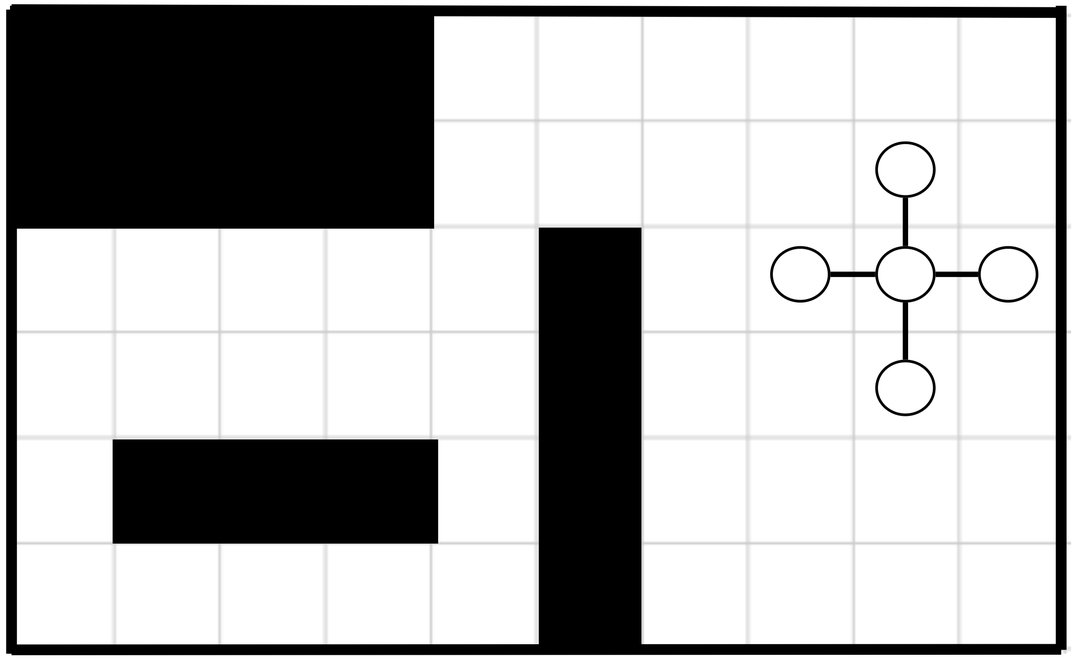
- **Graph Generation**
  - The space can be uniformly discretized into cells, each of which can be occupied by the robot.



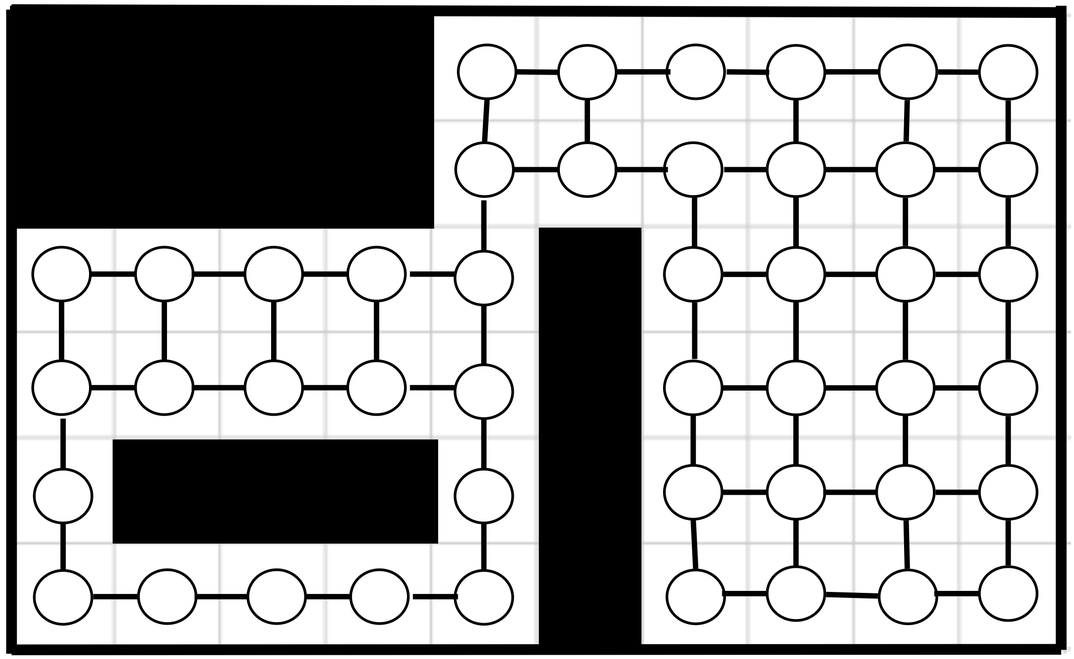
- **Graph Generation**
  - The cells can be described as free or occupied.



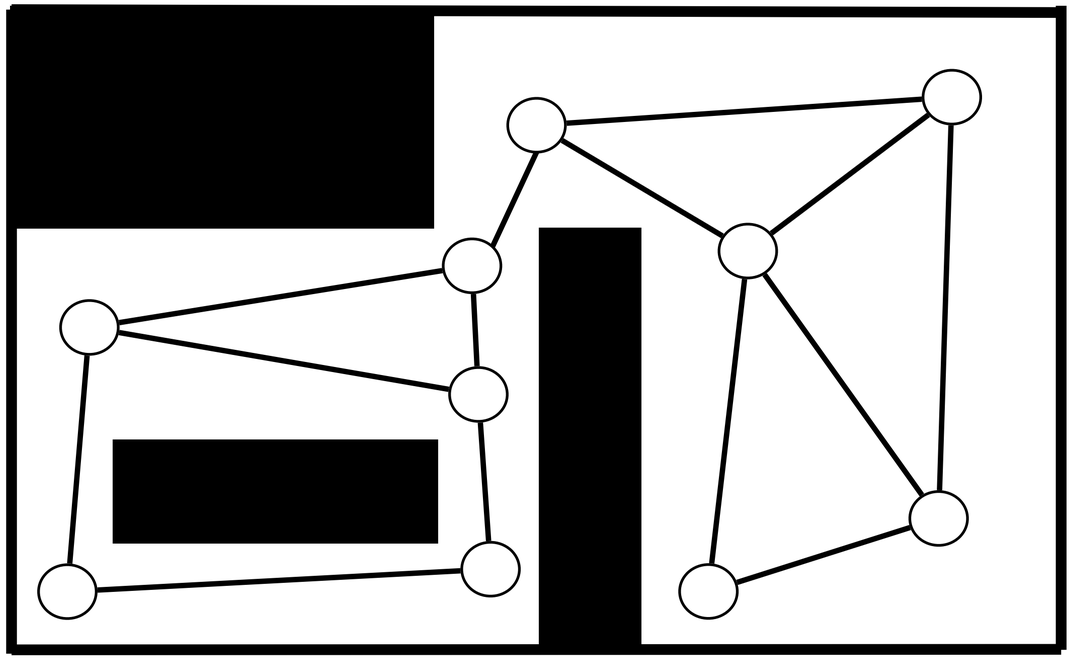
- **Graph Generation**
  - Each cell in the uniform decomposition corresponds to a node connected to adjacent nodes through edges.



- **Graph Generation**
  - A graph can be formed that is the collection of the nodes and edges.



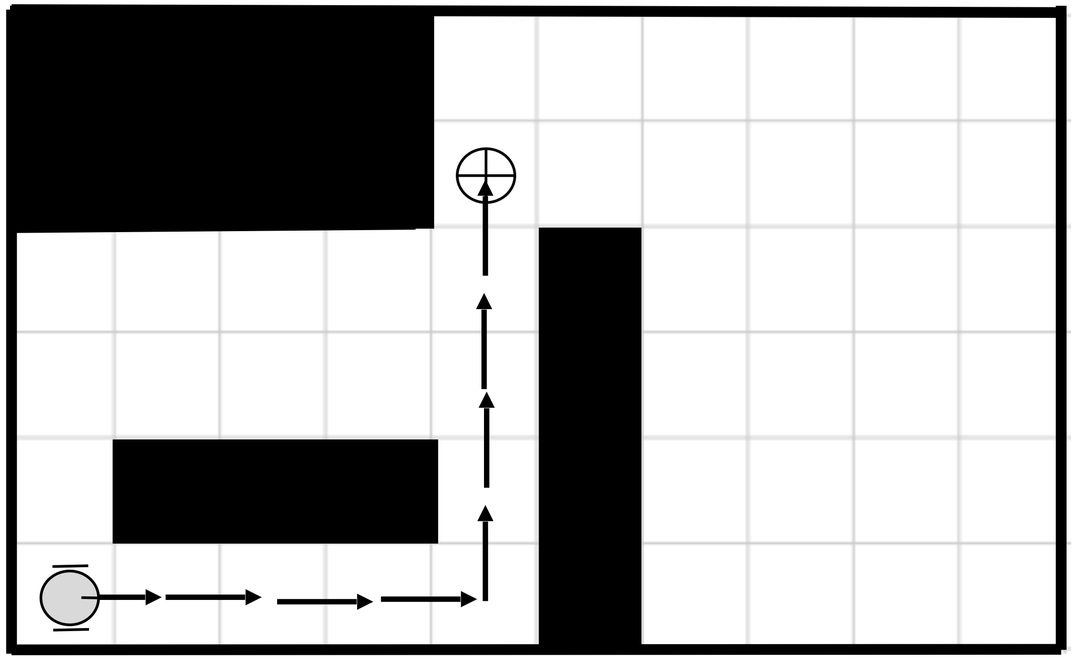
- **Graph Generation**
  - Graphs sometimes have fewer nodes and edges, but similar coverage of a space can be beneficial.



Agenda

1. The Graph
2. **Graph Search**
3. Breadth First Search
4. Depth First Search

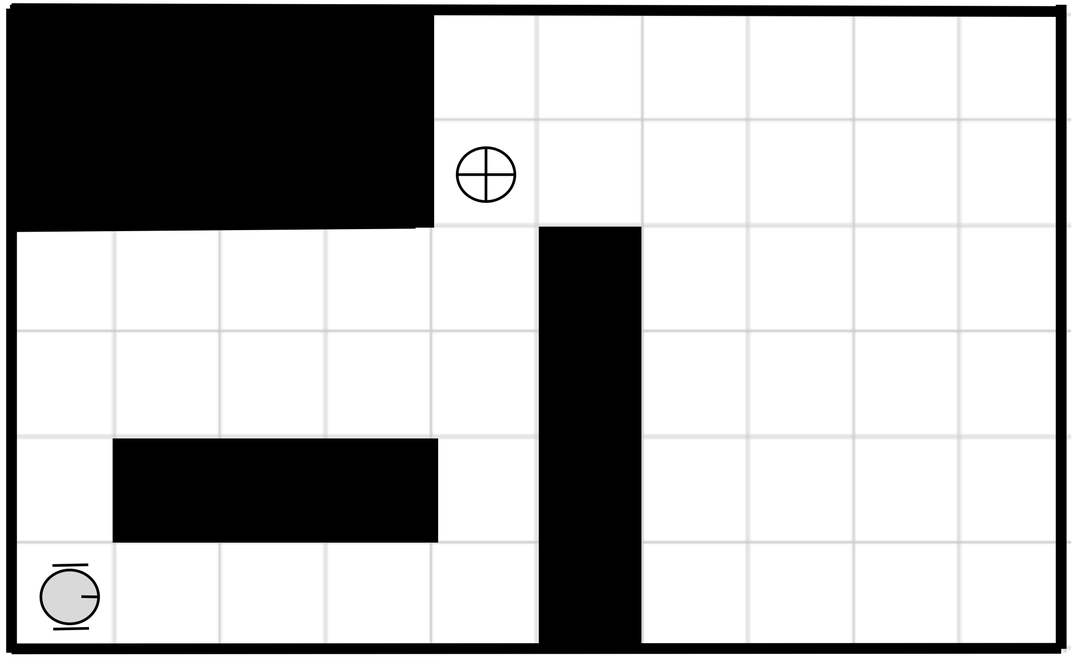
- How do we find a path from start to goal?



ROB-UY 3303

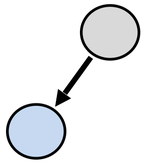
Robot Motion & Planning

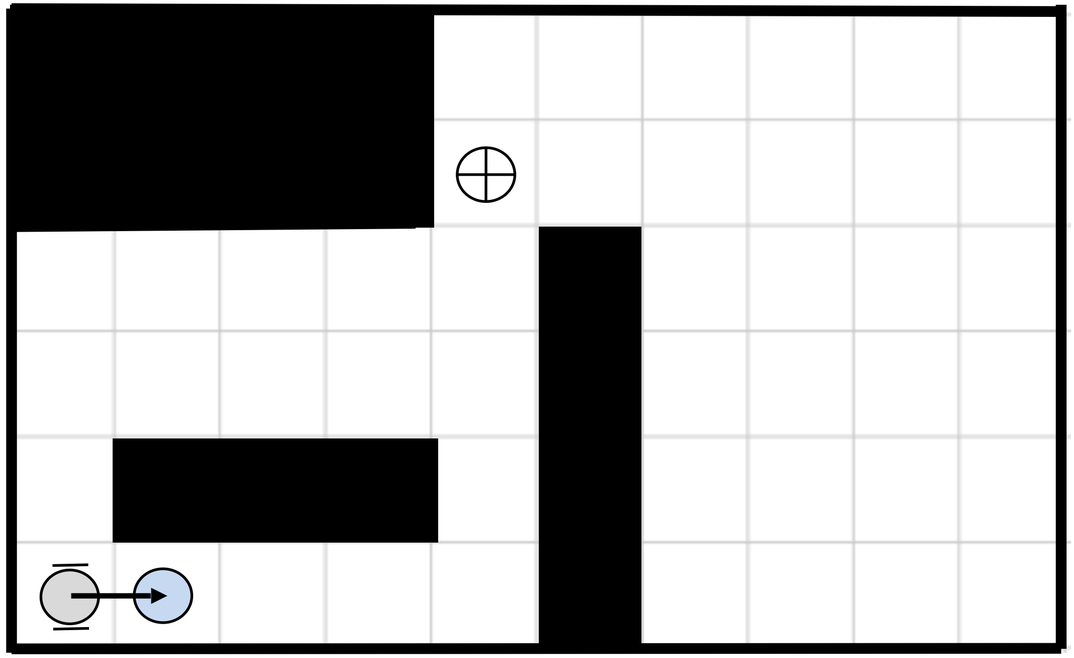


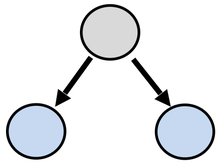


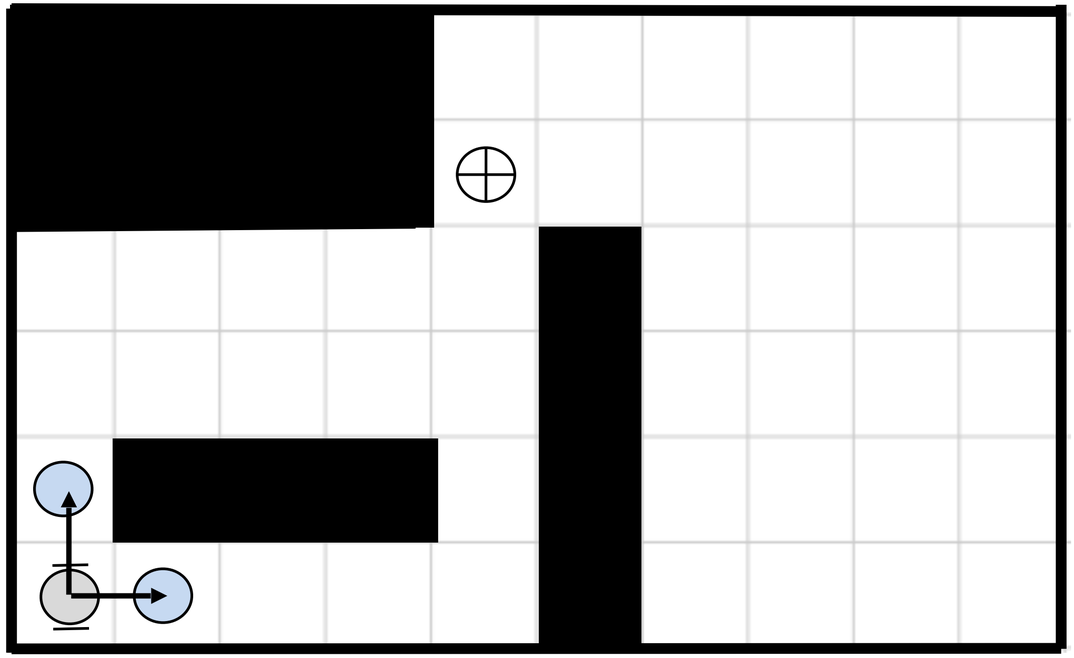
ROB-UY 3303

Robot Motion & Planning



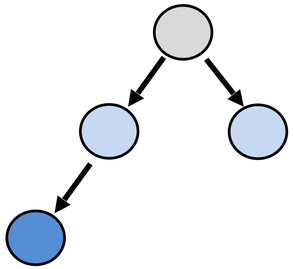


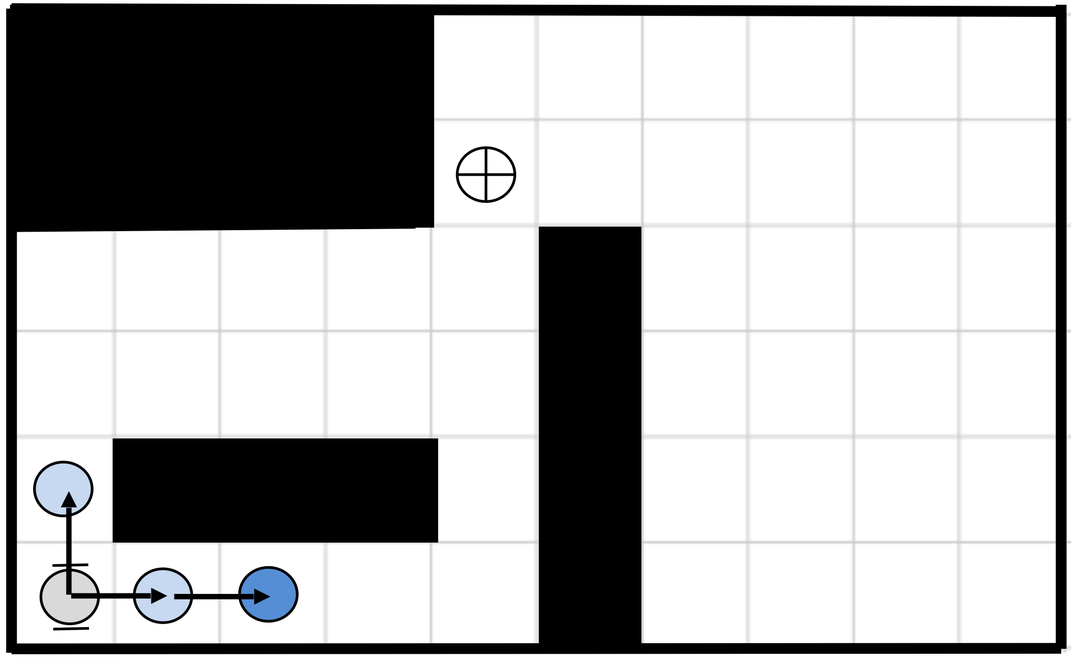




ROB-UY 3303

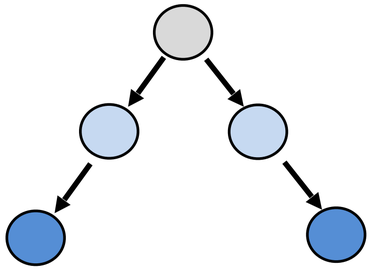
Robot Motion & Planning

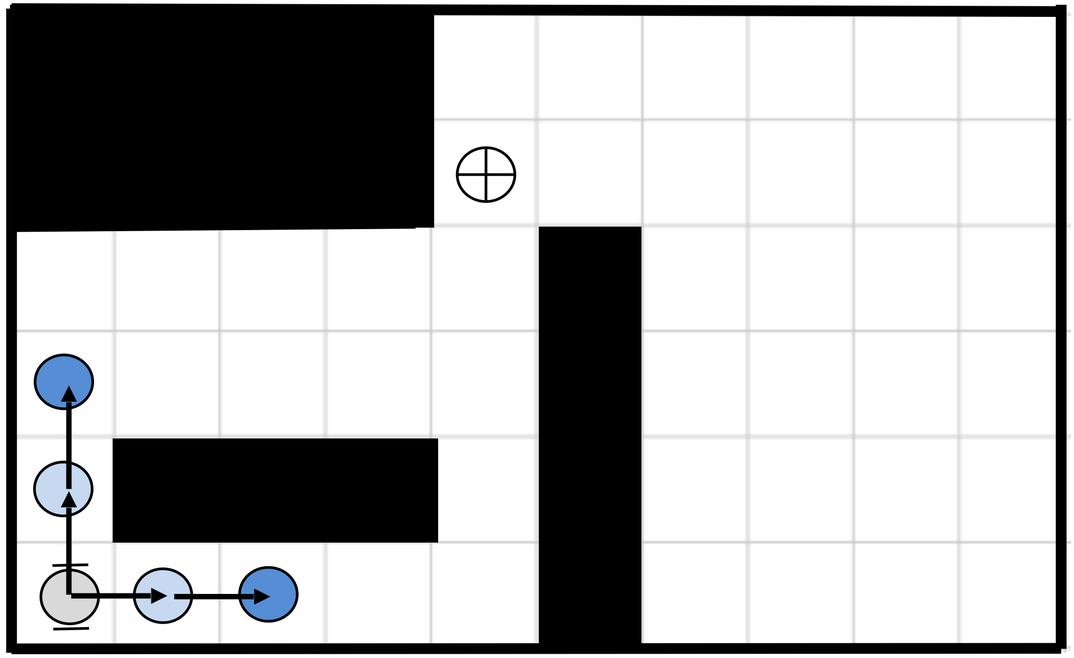




ROB-UY 3303

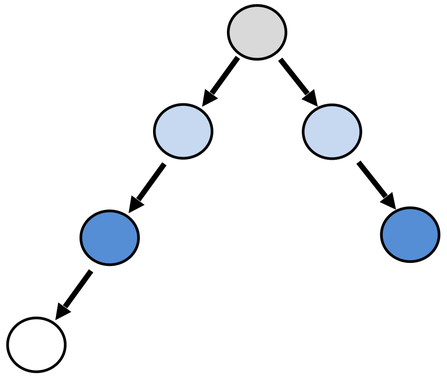
Robot Motion & Planning

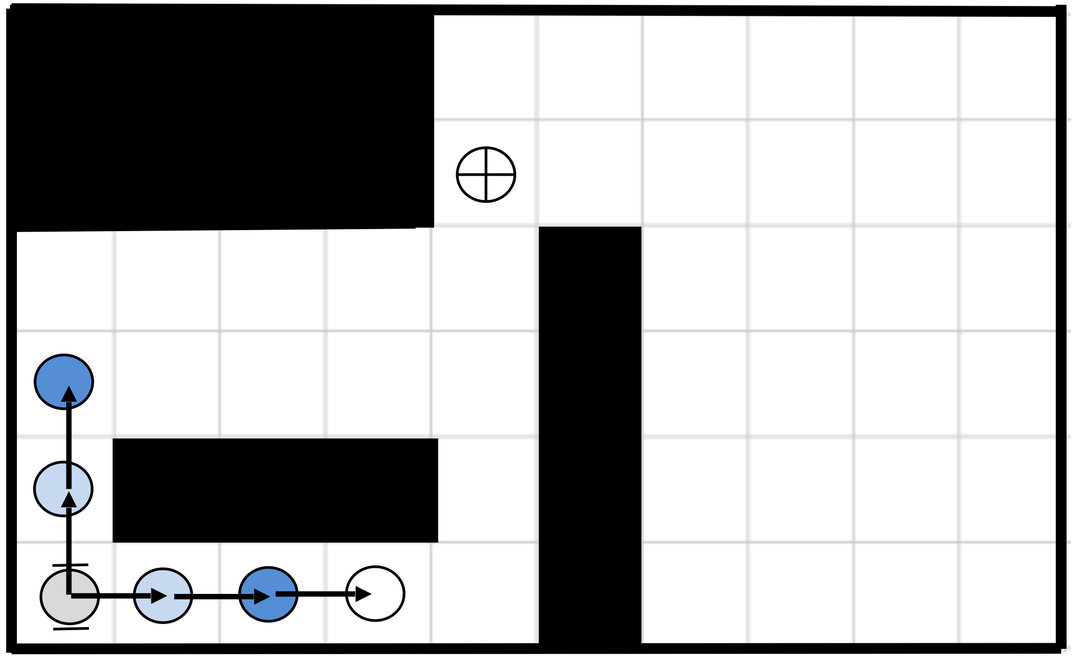




ROB-UY 3303

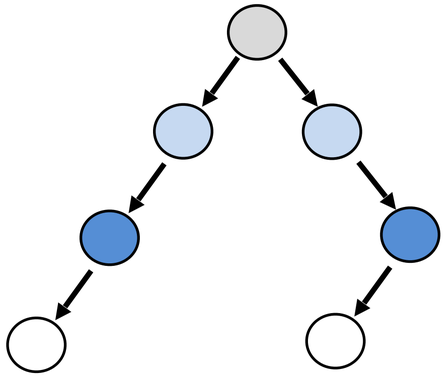
Robot Motion & Planning

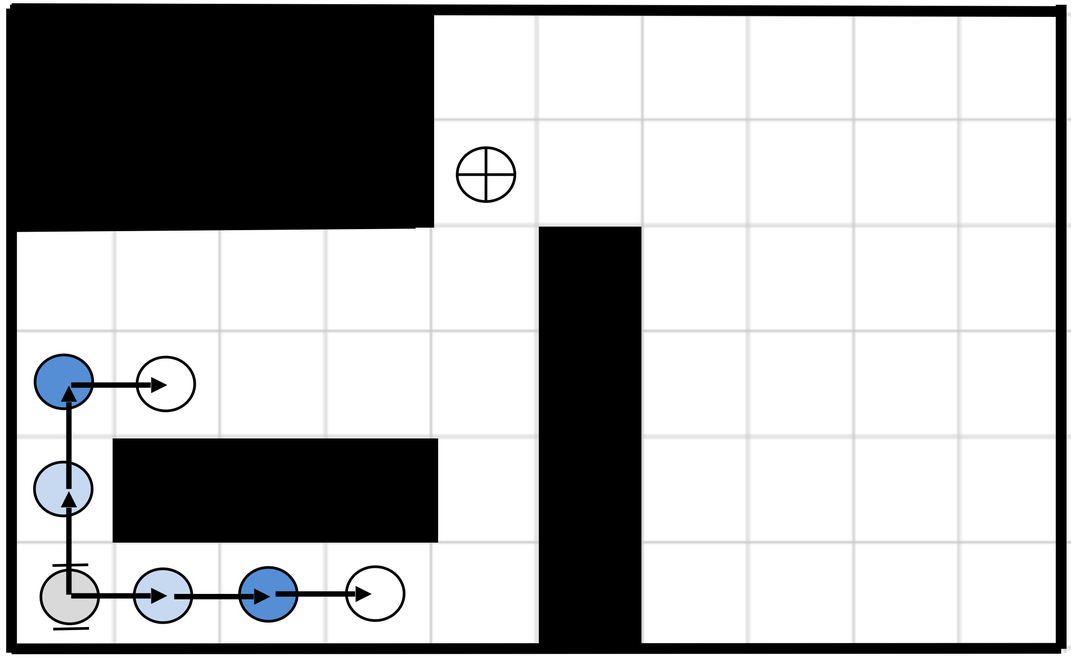




ROB-UY 3303

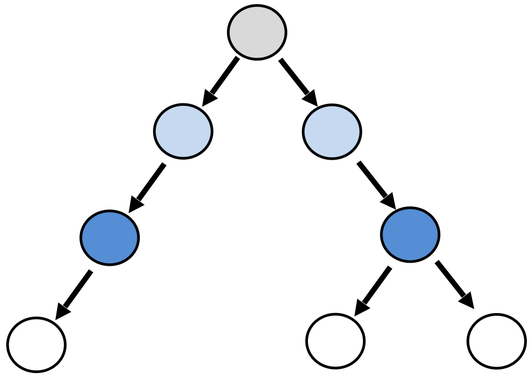
Robot Motion & Planning

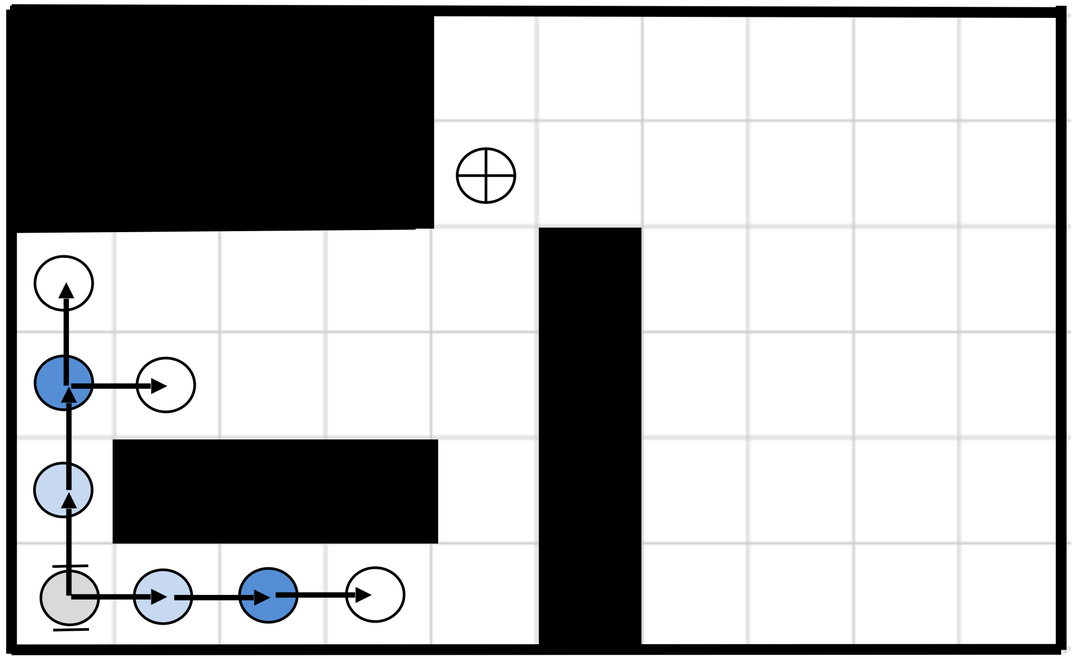




ROB-UY 3303

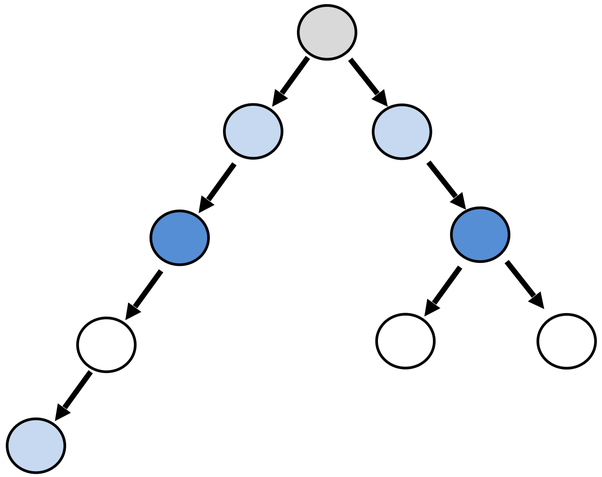
Robot Motion & Planning

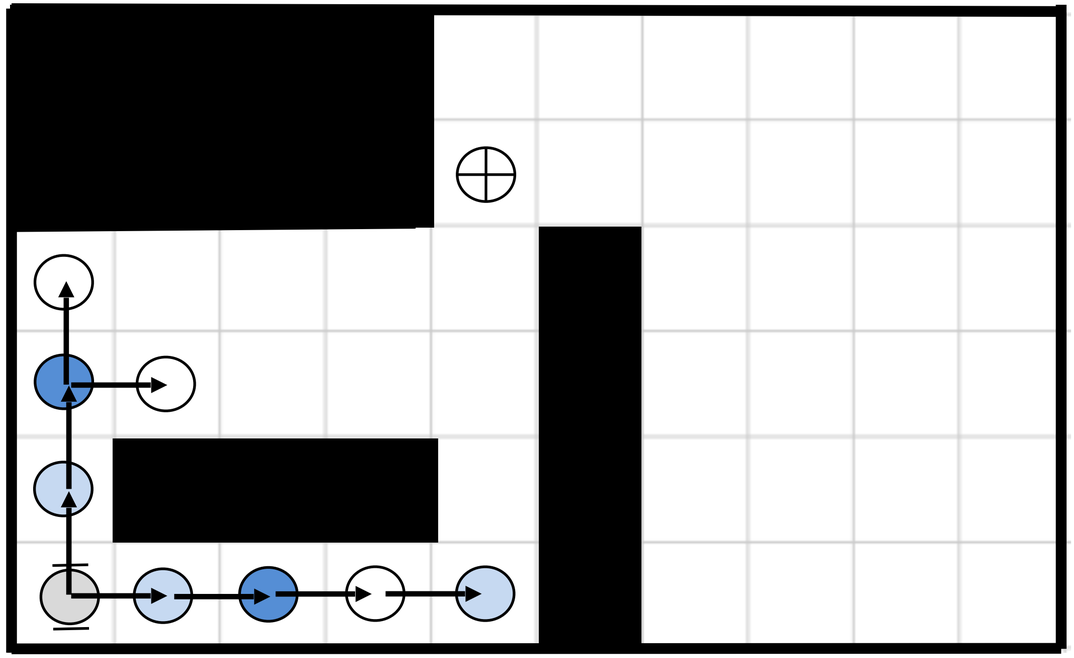




ROB-UY 3303

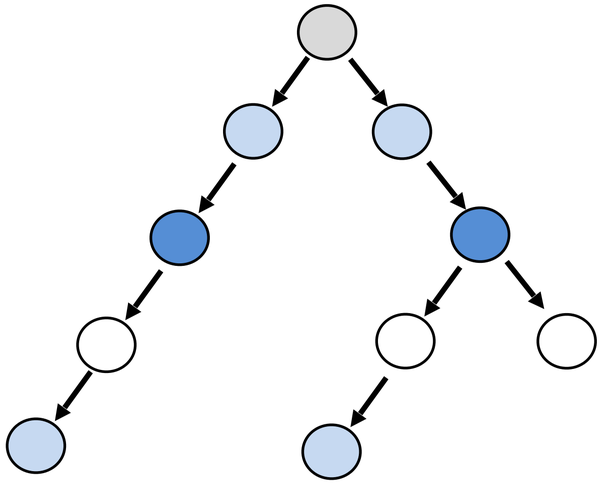
Robot Motion & Planning

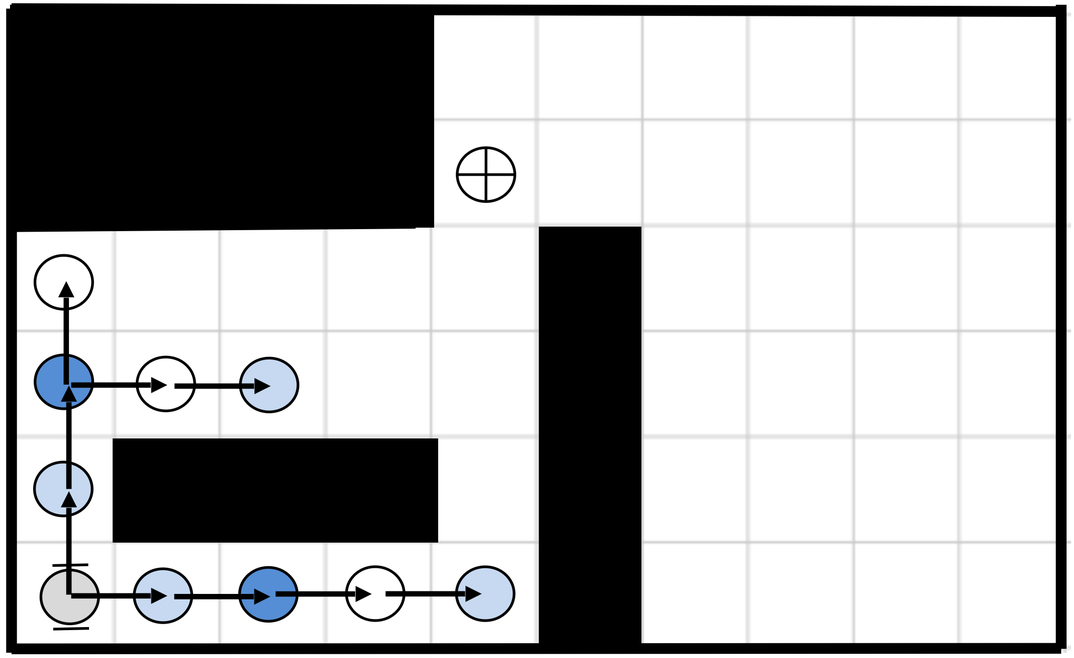


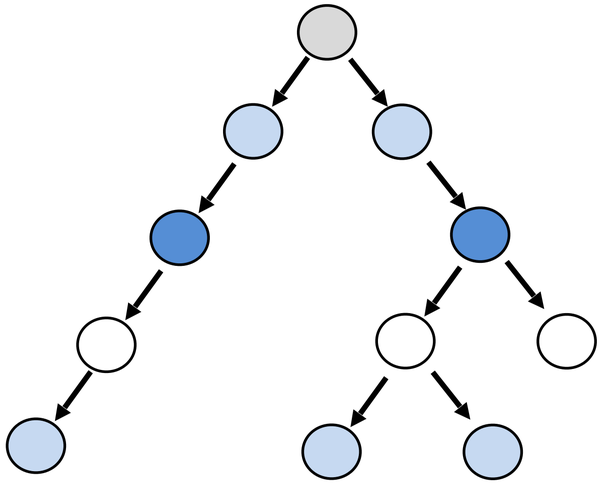


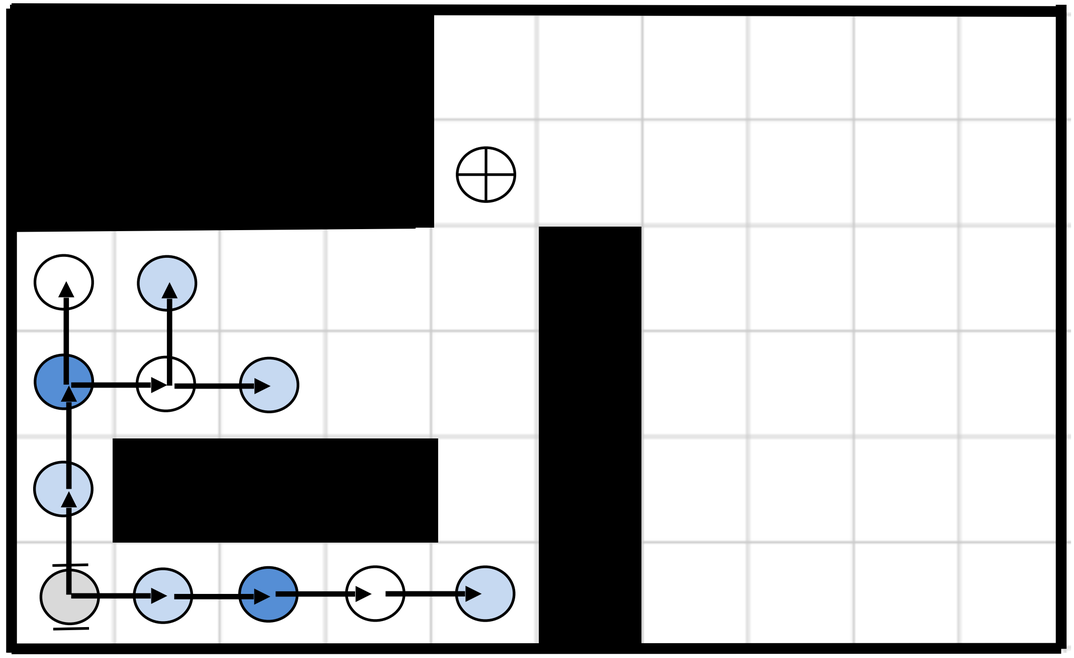
ROB-UY 3303

Robot Motion & Planning



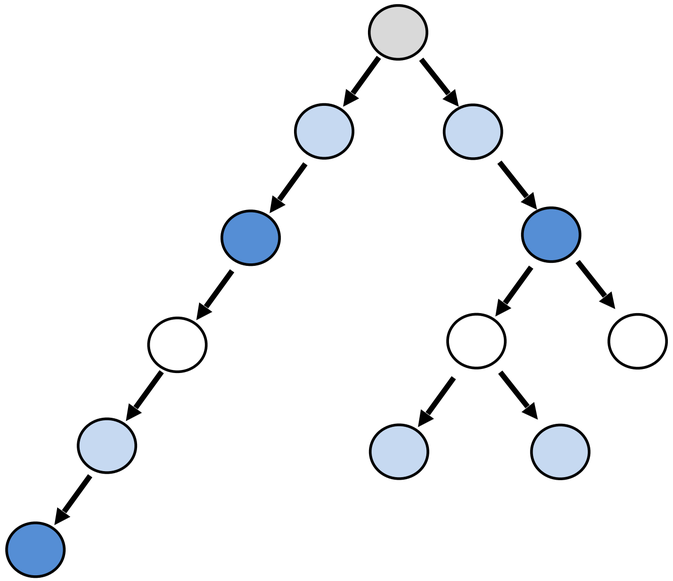


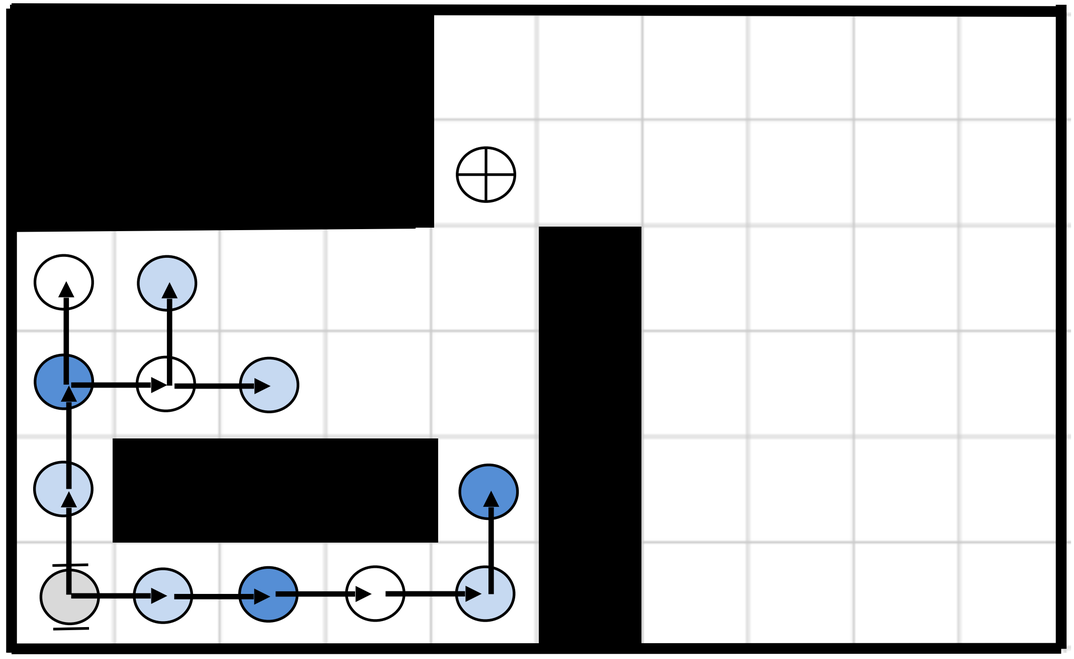




ROB-UY 3303

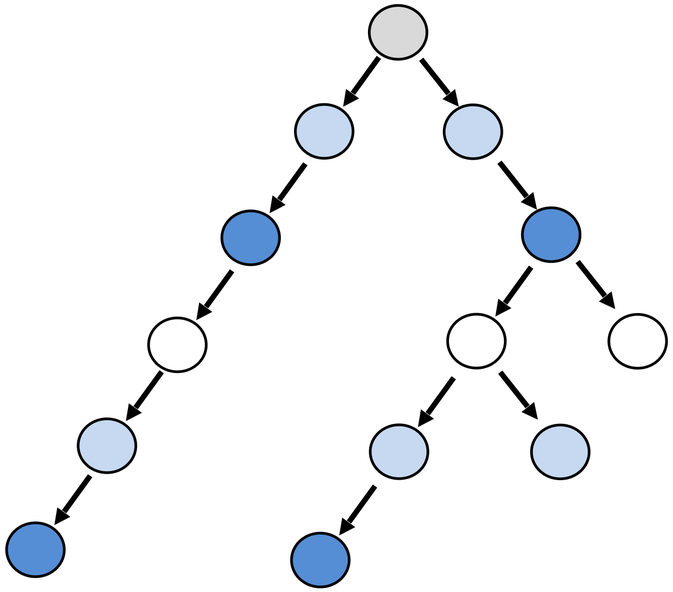
Robot Motion & Planning

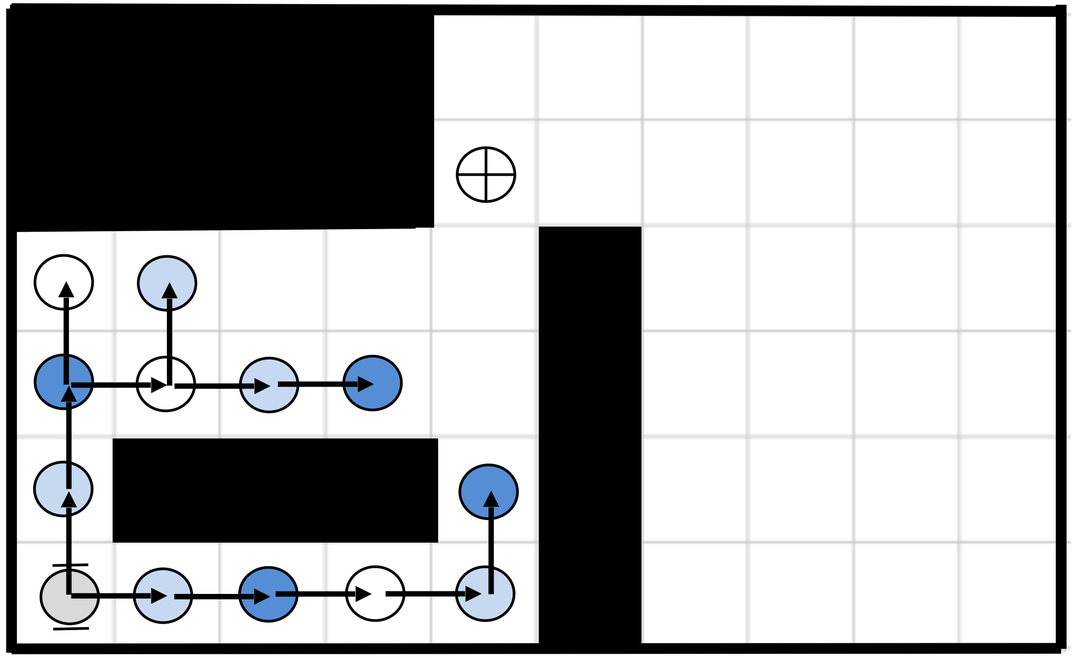




ROB-UY 3303

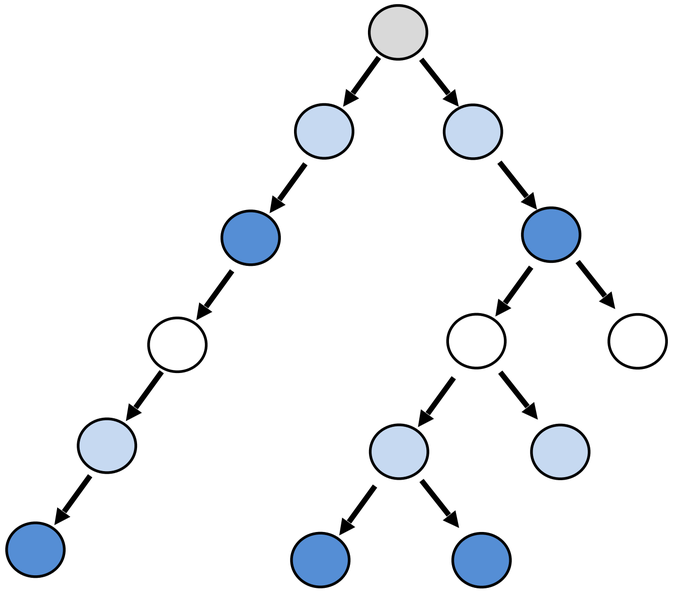
Robot Motion & Planning

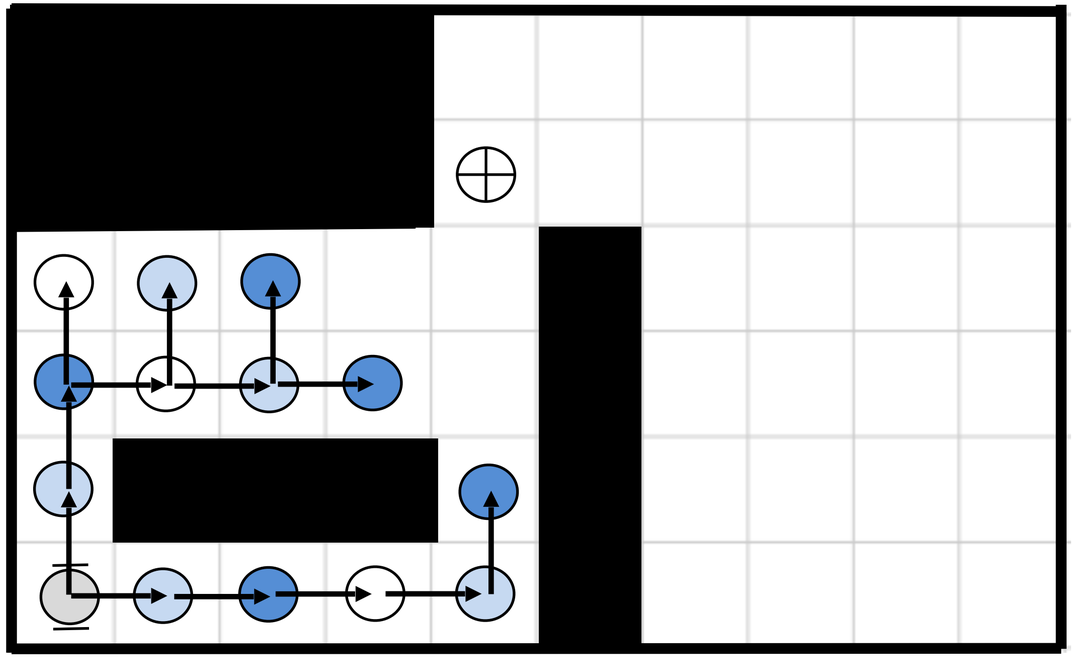




ROB-UY 3303

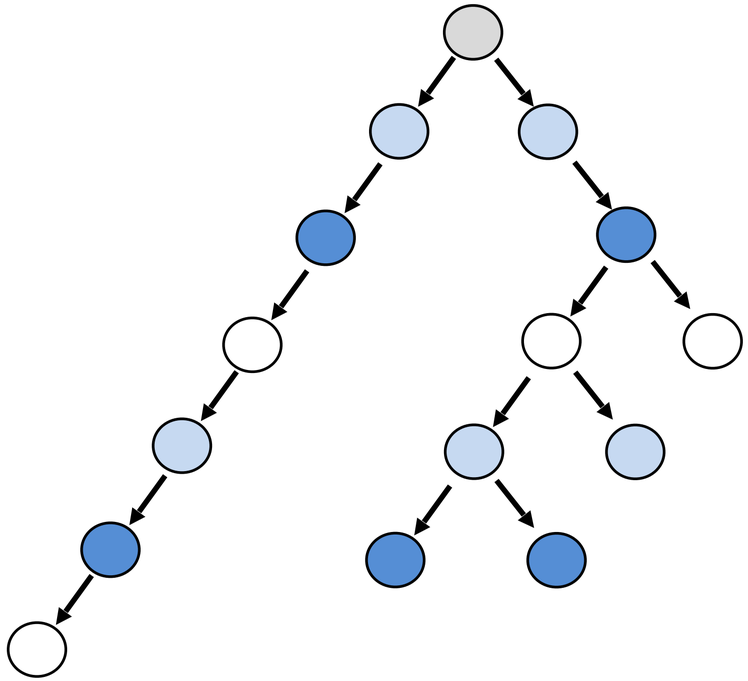
Robot Motion & Planning

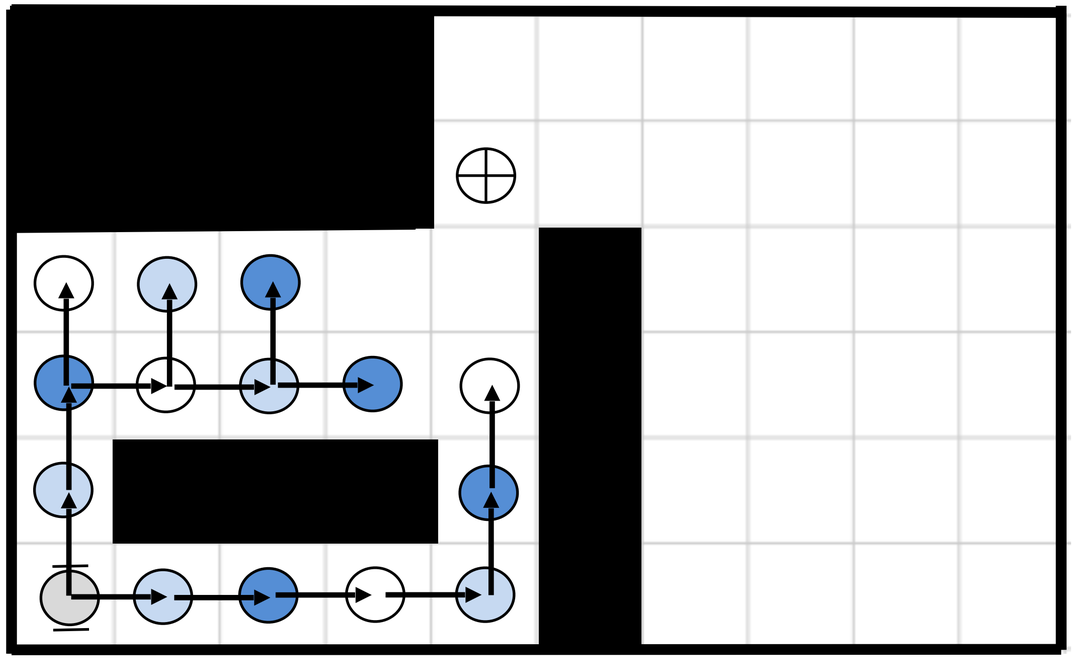




ROB-UY 3303

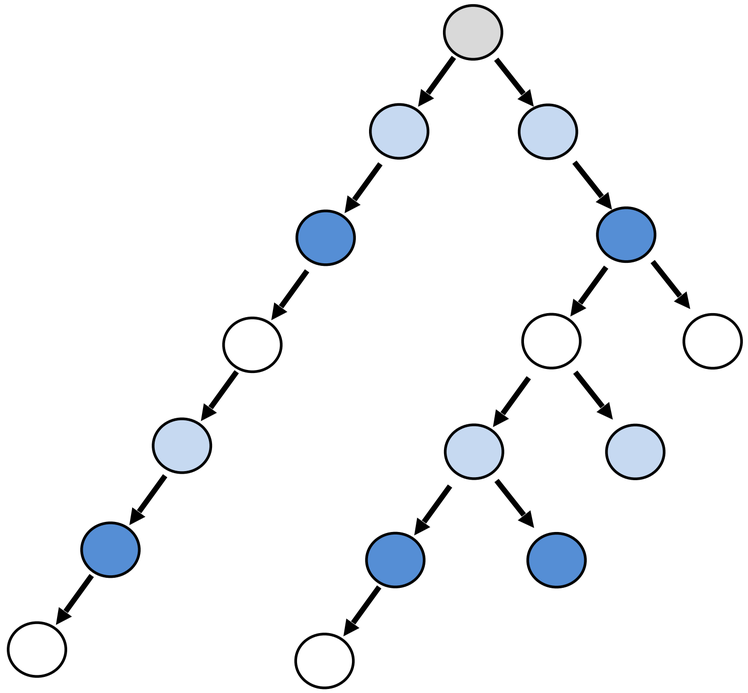
Robot Motion & Planning

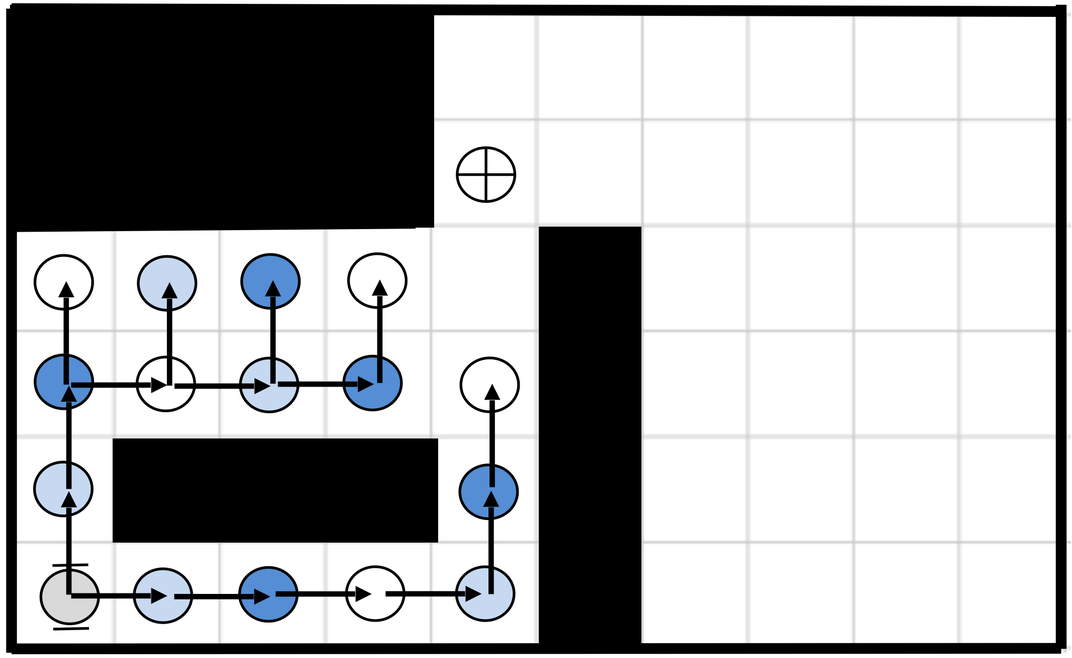




ROB-UY 3303

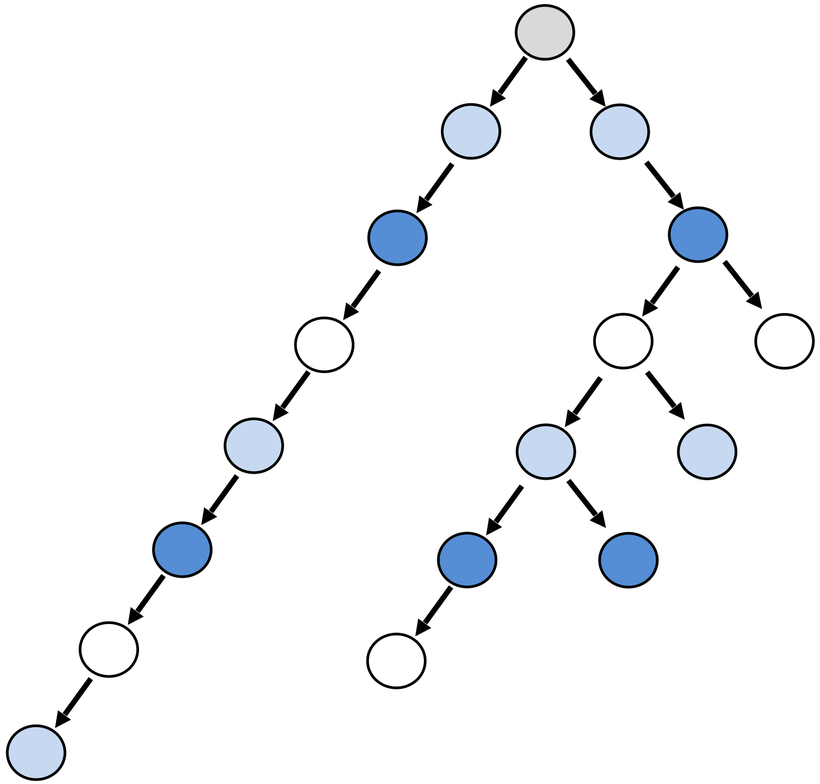
Robot Motion & Planning

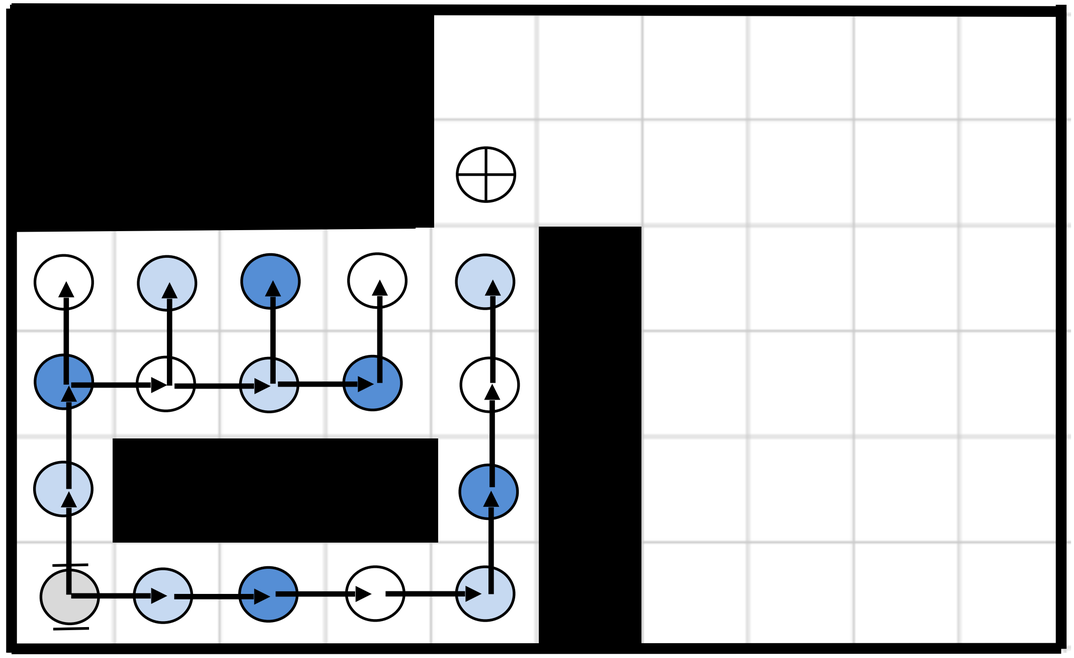


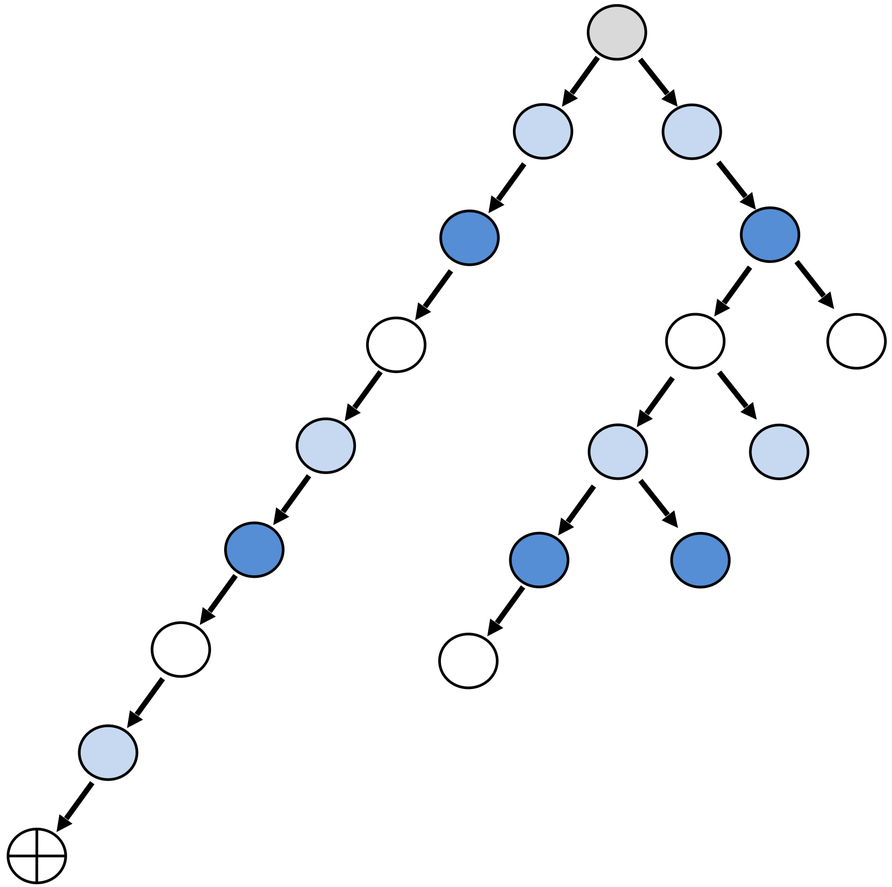


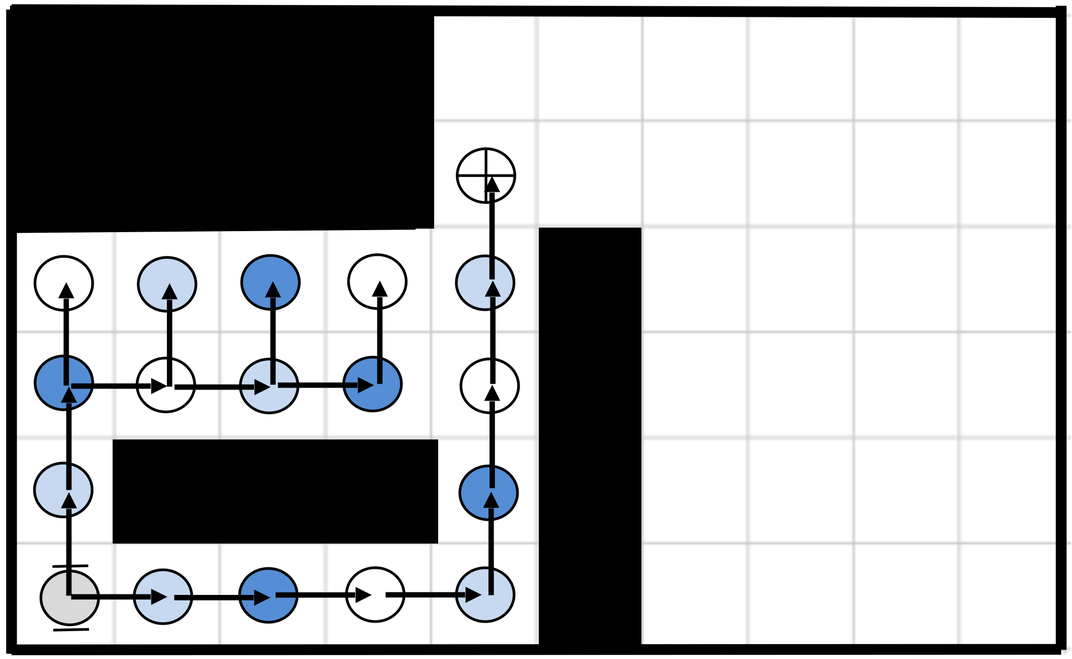
ROB-UY 3303

Robot Motion & Planning



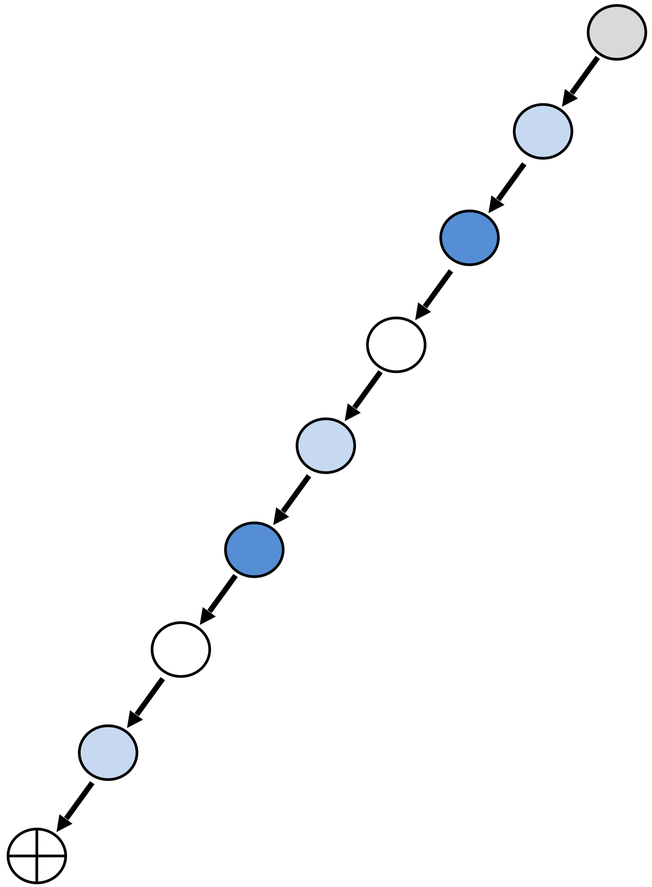


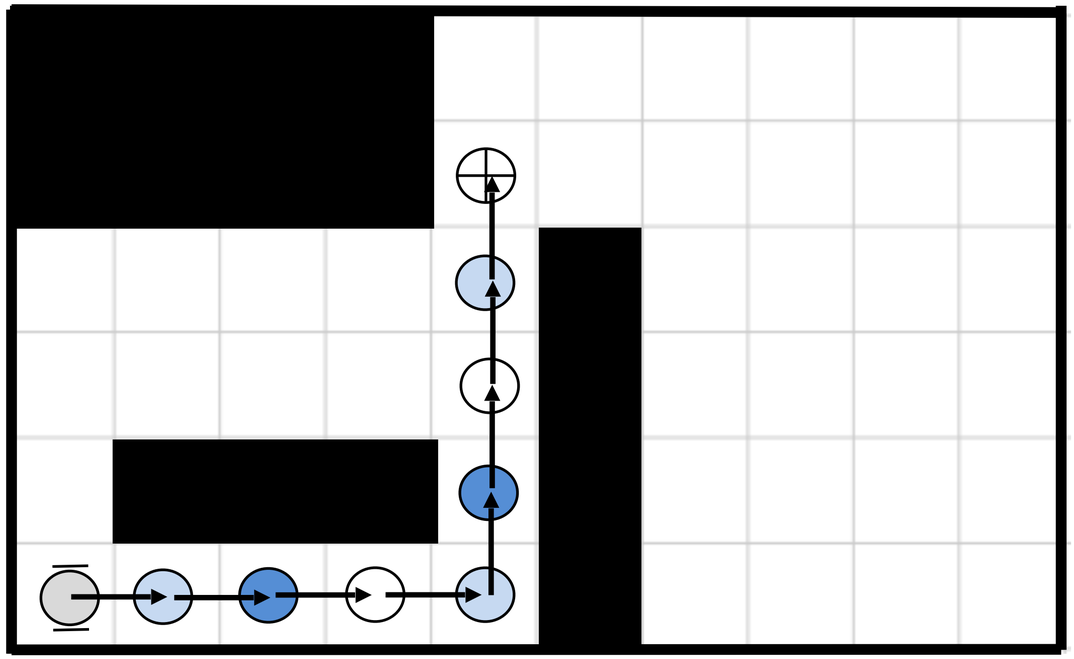




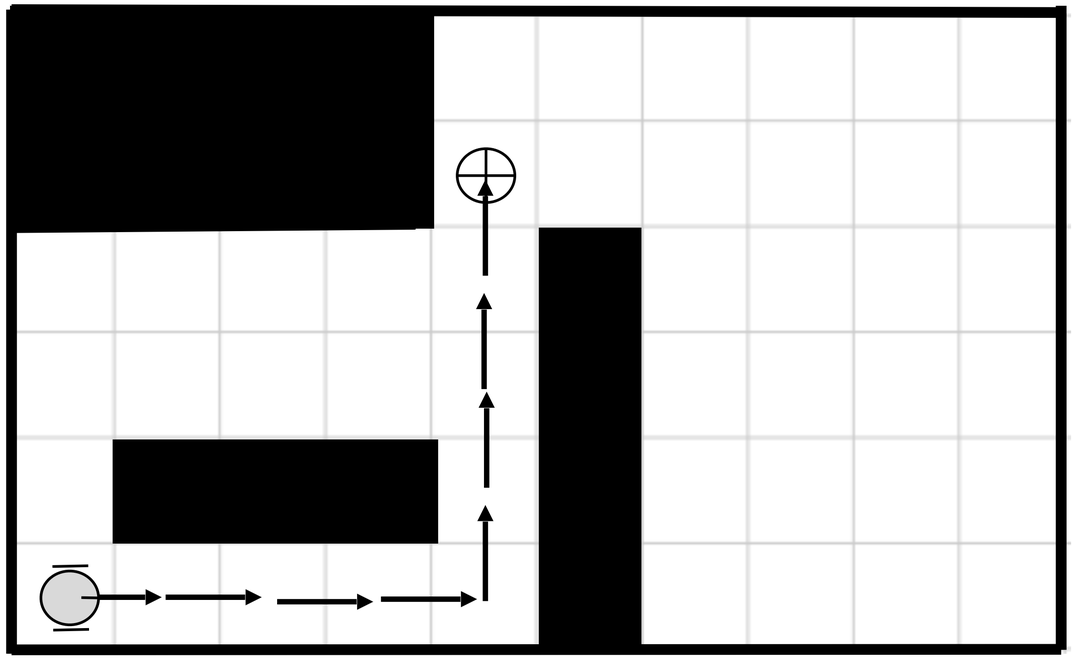
ROB-UY 3303

Robot Motion & Planning



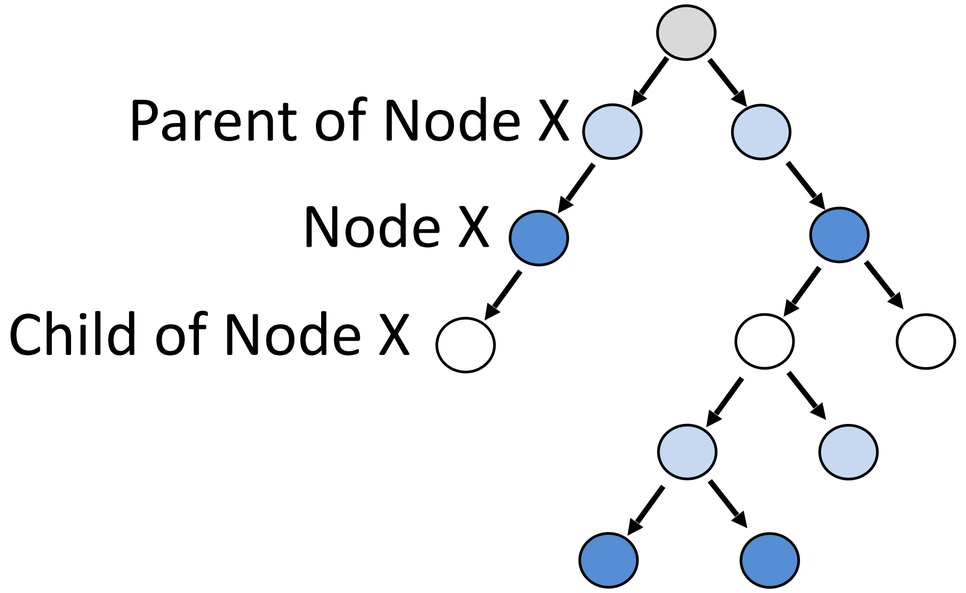


- **How do we find a path from start to goal?**
  - Let us add some language to help



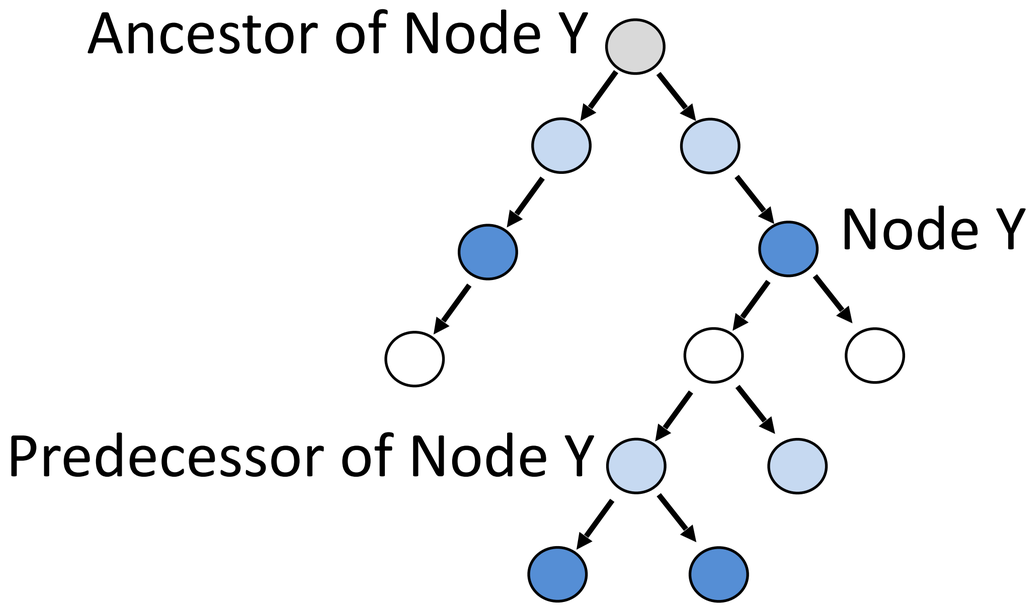
Tree Nodes

- Nodes can have parent nodes and child nodes
- Nodes can have many children
- Nodes can only have 1 parent

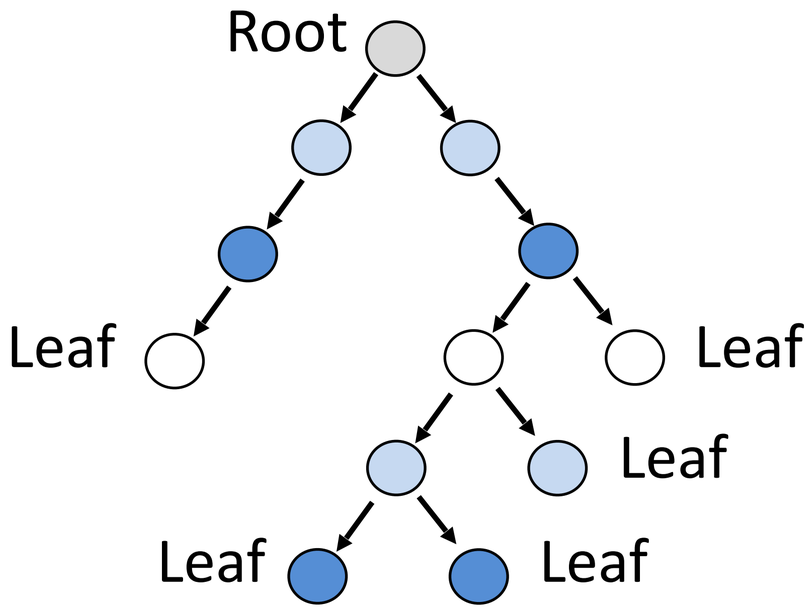


Tree Nodes

- Nodes can have many ancestors
- Nodes can have many predecessors

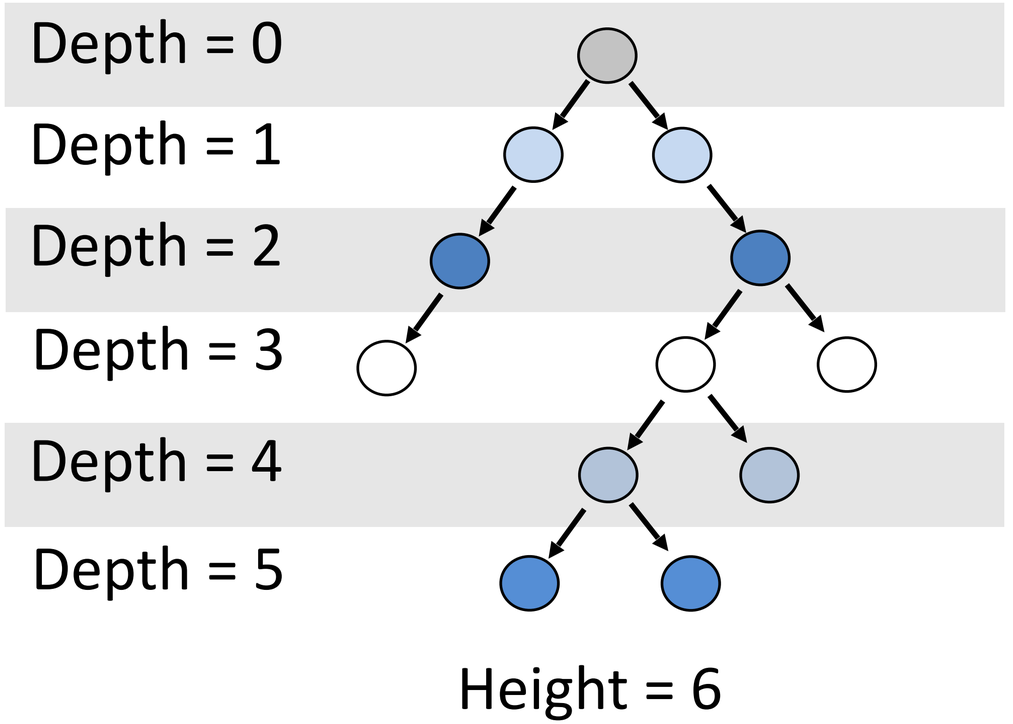


- **Tree Nodes**
  - The root node has no parent
  - Leaf nodes have no children

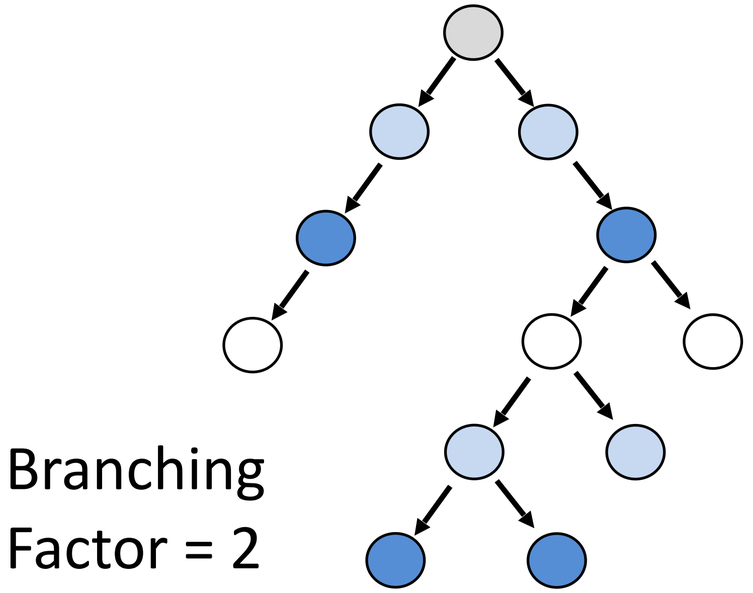


Tree Structure

- Each tree node has its own depth level:
  - The number of edges between it and the root
- The tree's height is the maximum depth of nodes in the tree + 1



- **Tree Structure**
  - The branching factor of a tree is the maximum number of children each node in the tree has.



General Graph Search Algorithms

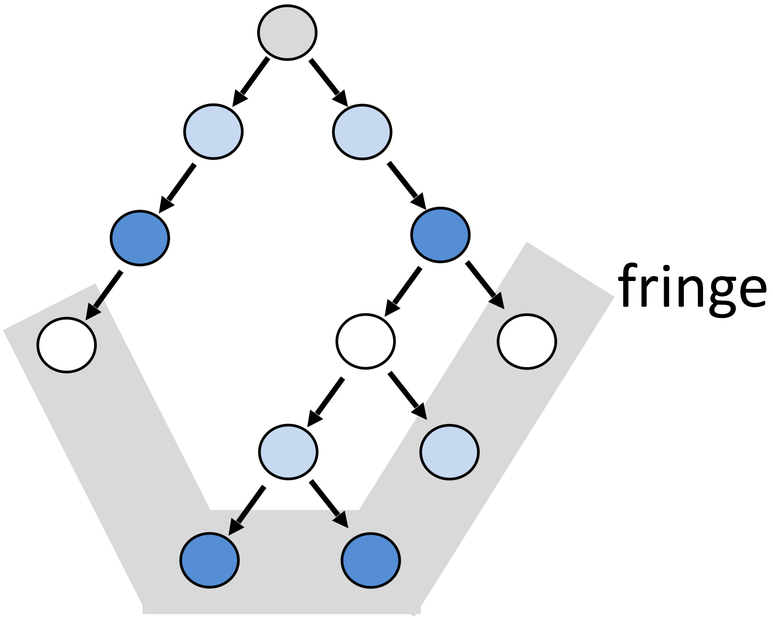
```
function GENERAL-SEARCH(problem, strategy)

     add root node to fringe

     loop do
          if fringe is empty
               return failure

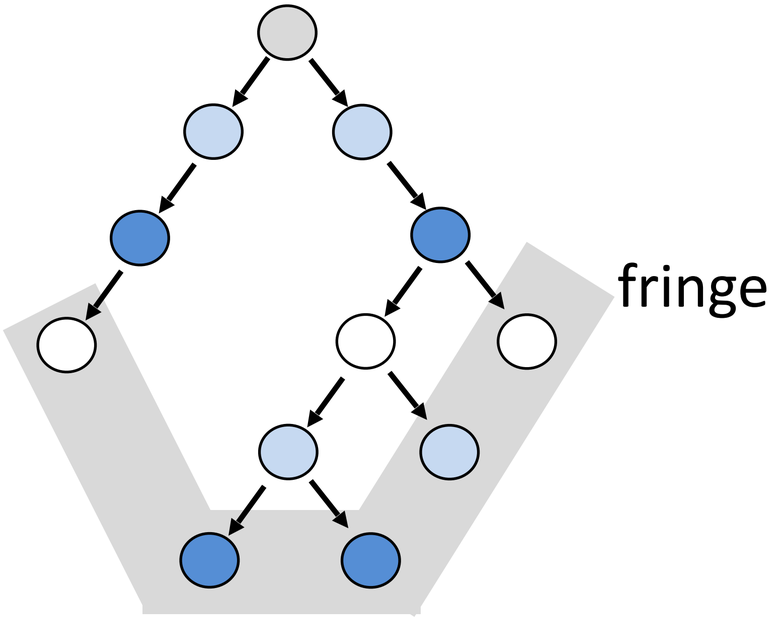
          pick node n from fringe via strategy

          if the node n contains a goal
               return path from root to n
          else
               add node n's children to fringe
     end
```



- **General Graph Search Algorithms**

  - The algorithm's strategy is used to determine **which node** on the fringe to expand from next
  - Algorithms differ mostly by their strategy

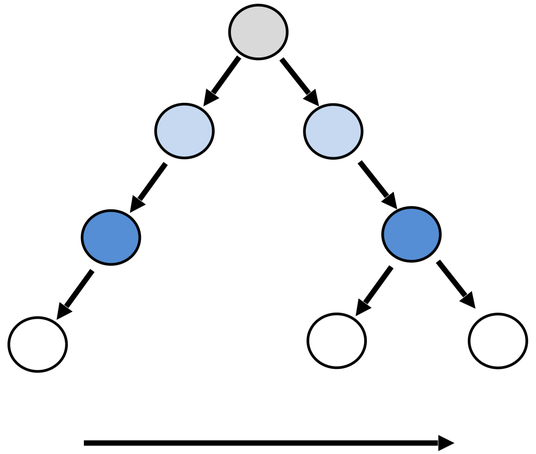


Agenda

1. The Graph
2. Graph Search
3. **Breadth First Search**
4. Depth First Search

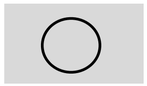
Breadth first search

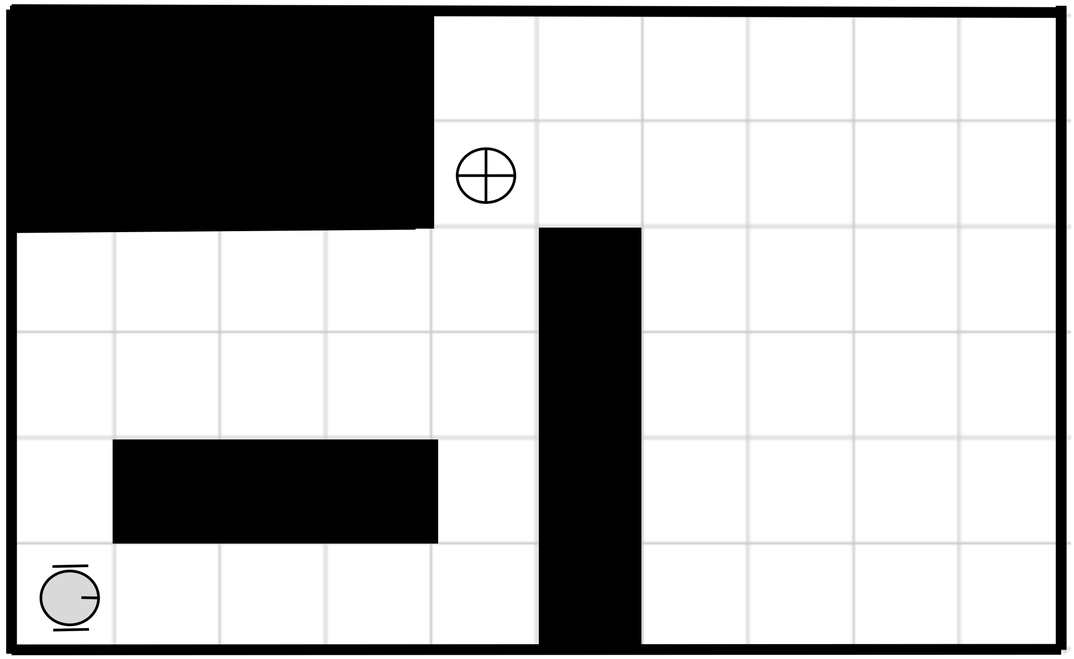
- Uses a strategy where nodes are expanded one depth level at a time.
  - i.e. All nodes of depth level d will be expanded before any node of depth level d+1



ROB-UY 3303

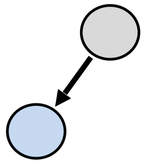
Robot Motion & Planning

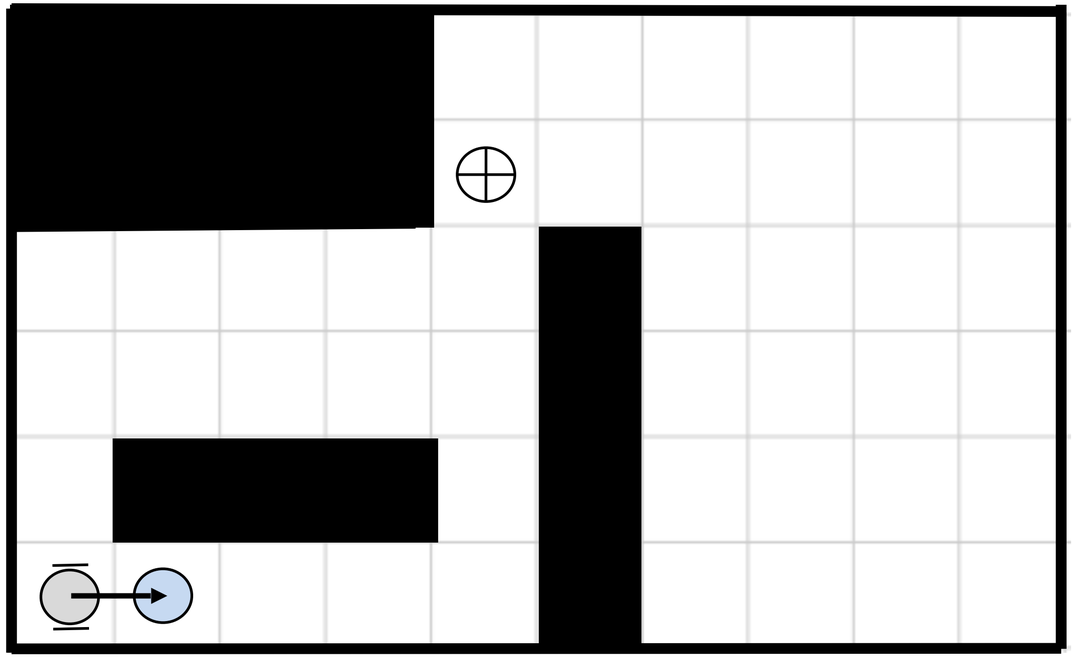




ROB-UY 3303

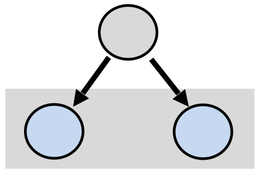
Robot Motion & Planning

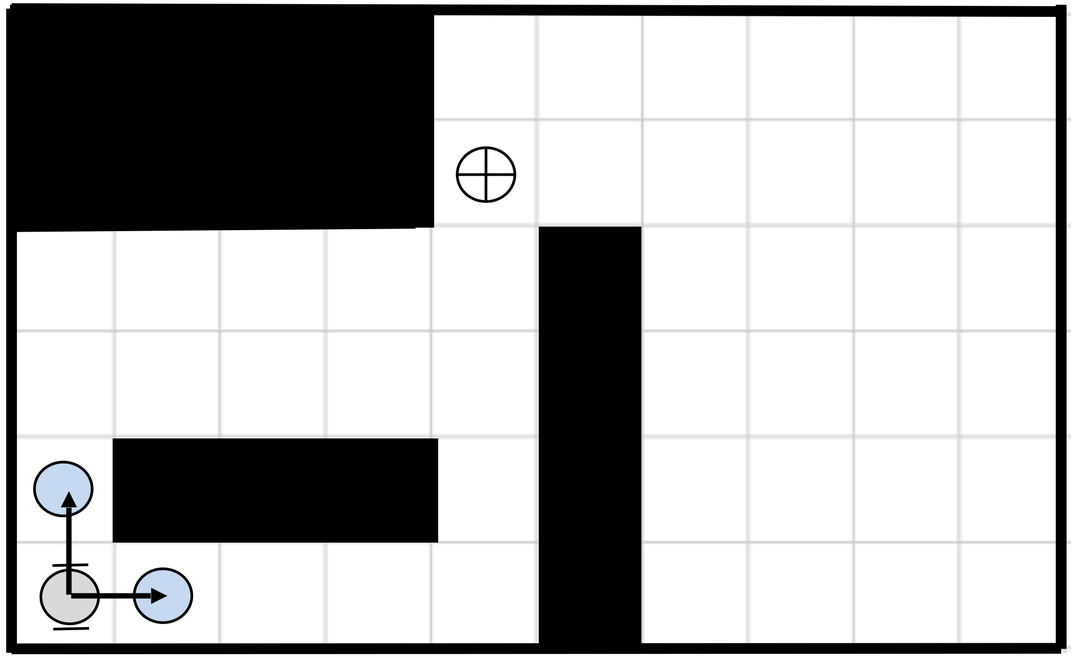




ROB-UY 3303

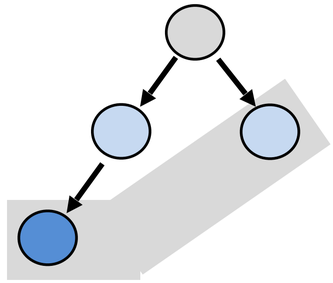
Robot Motion & Planning

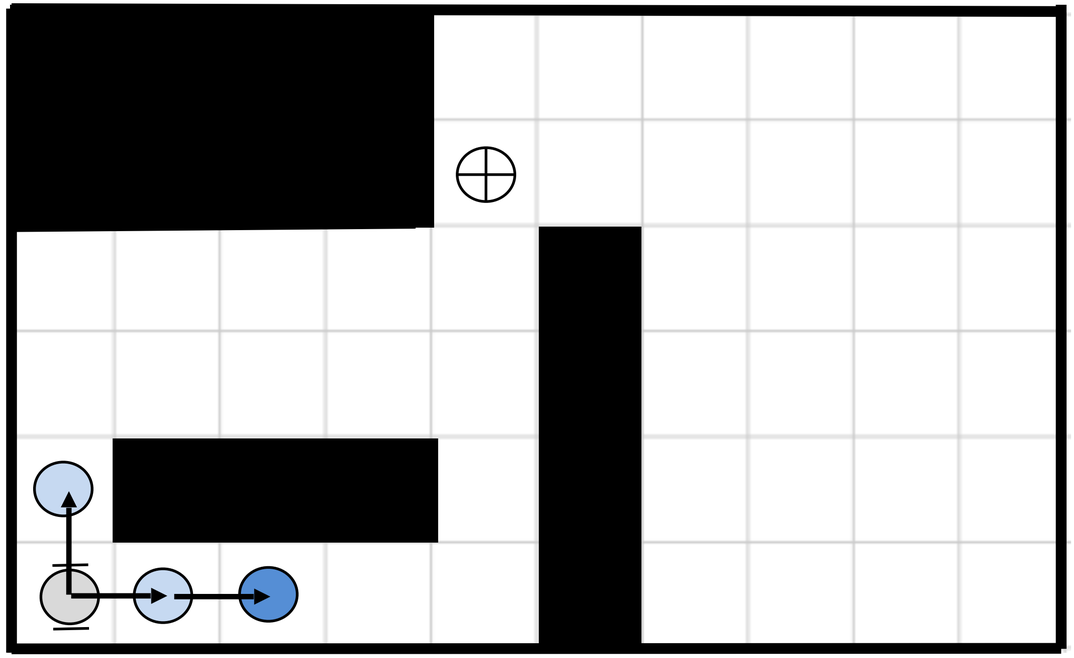




ROB-UY 3303

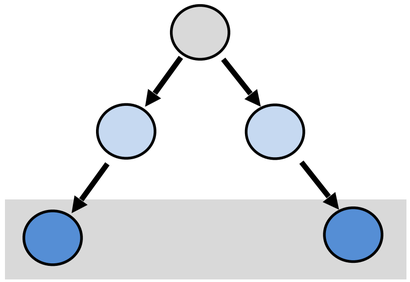
Robot Motion & Planning

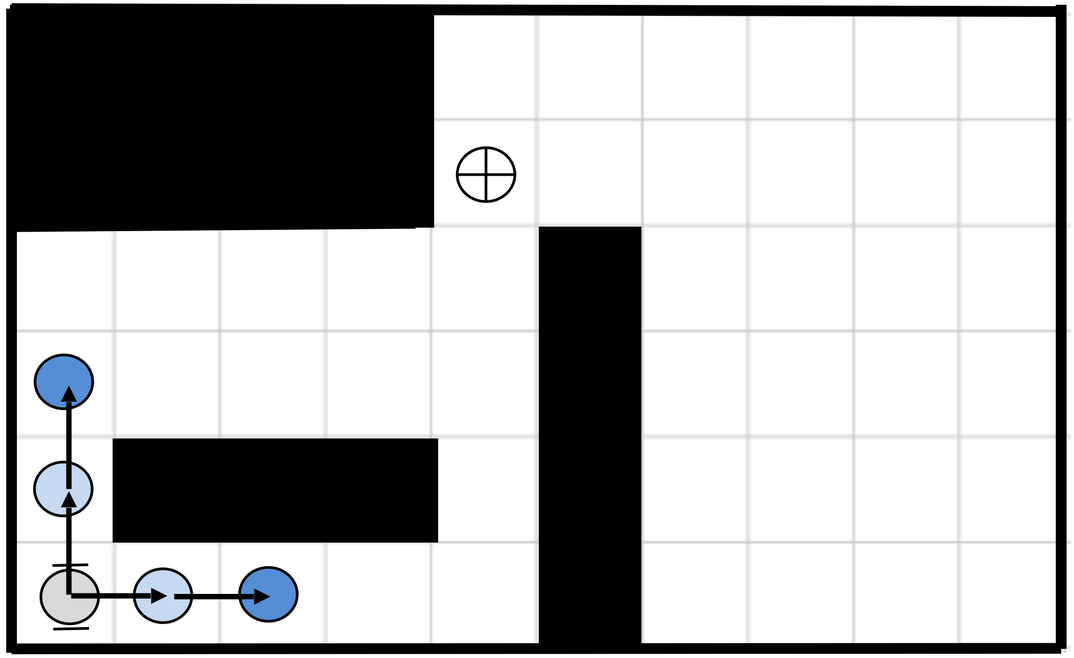




ROB-UY 3303

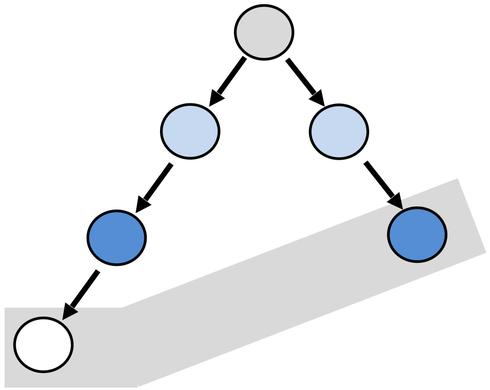
Robot Motion & Planning

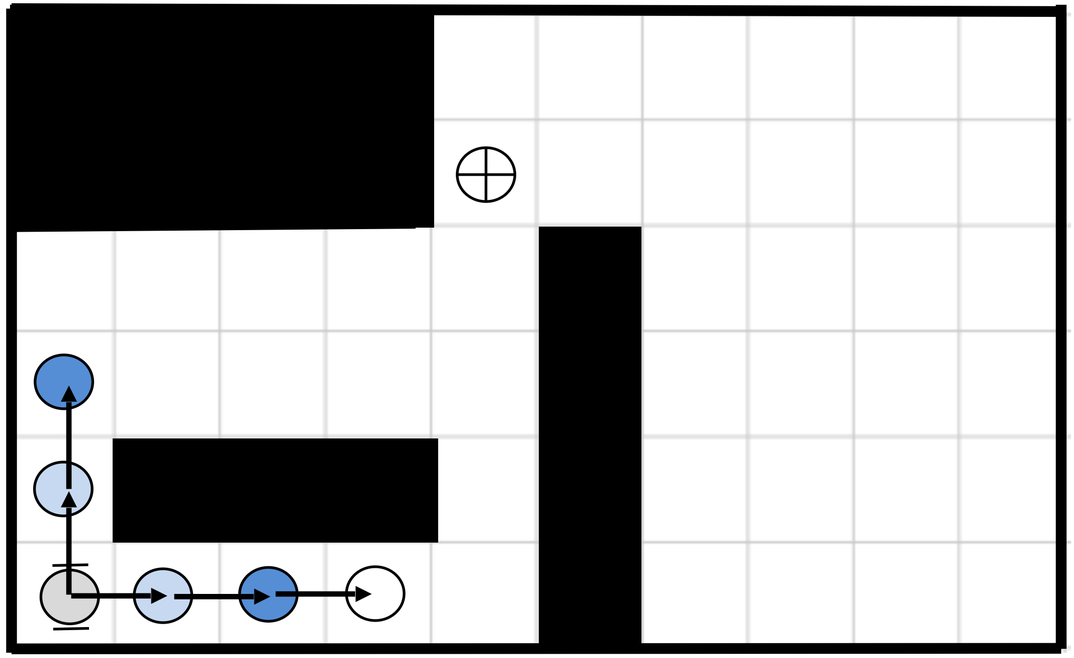




ROB-UY 3303

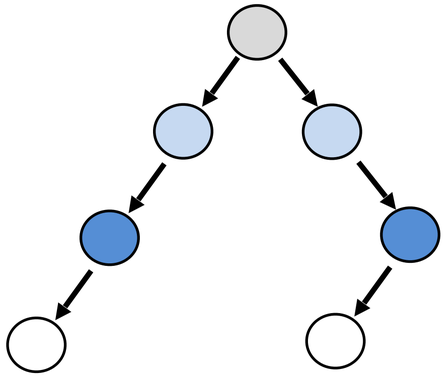
Robot Motion & Planning

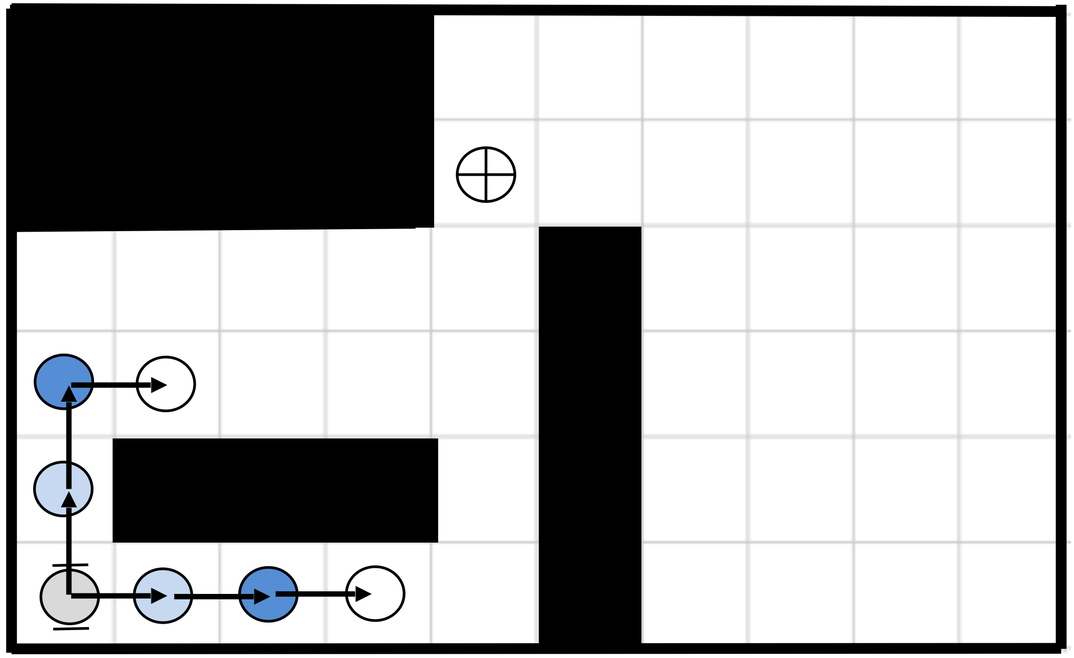




ROB-UY 3303

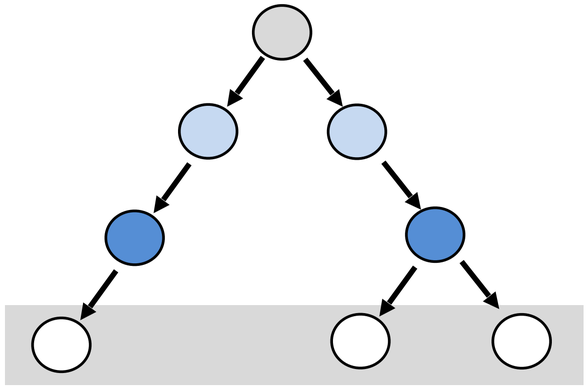
Robot Motion & Planning

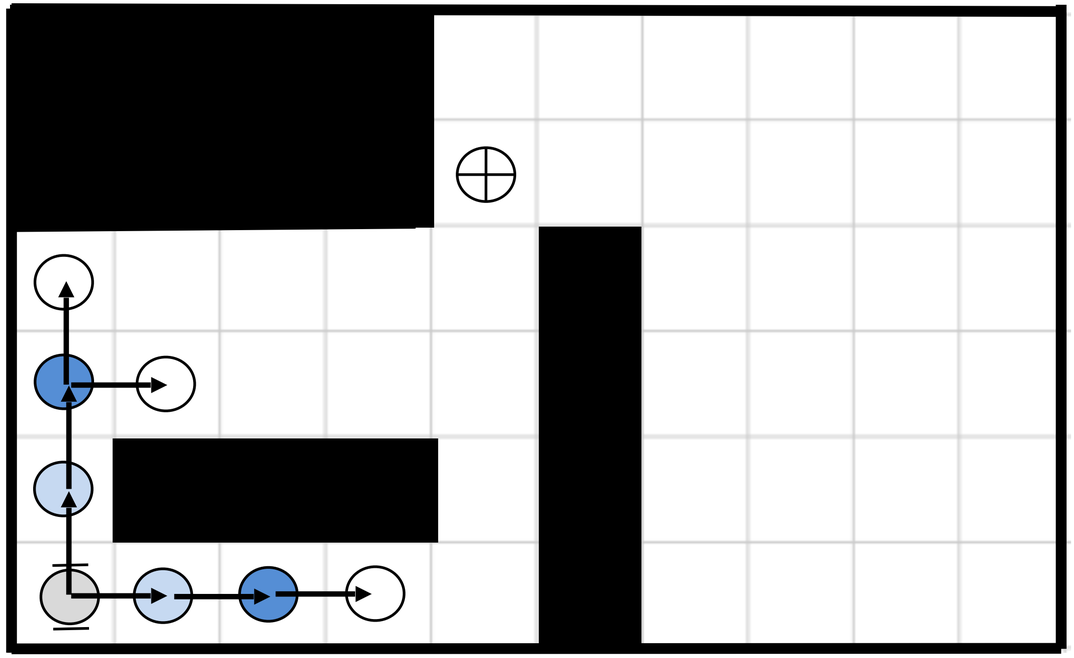




ROB-UY 3303

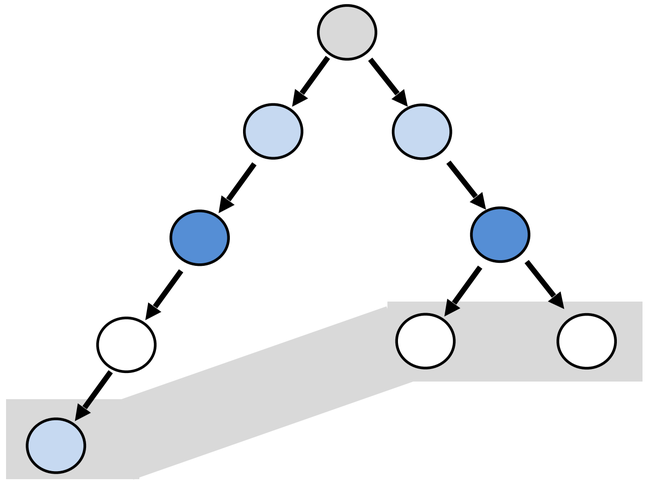
Robot Motion & Planning

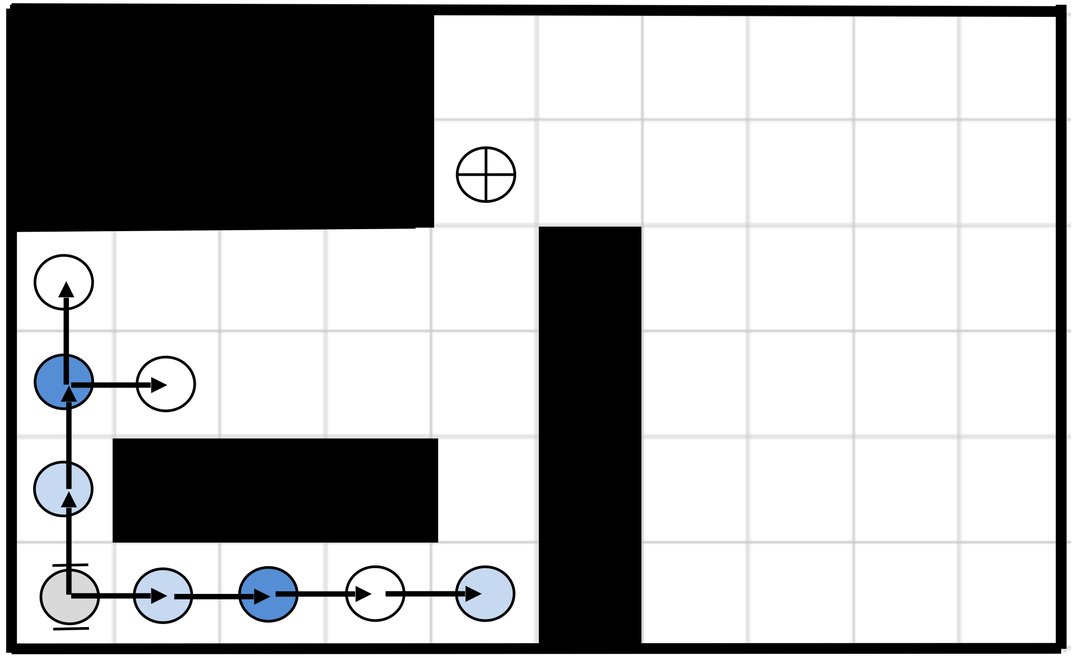




ROB-UY 3303

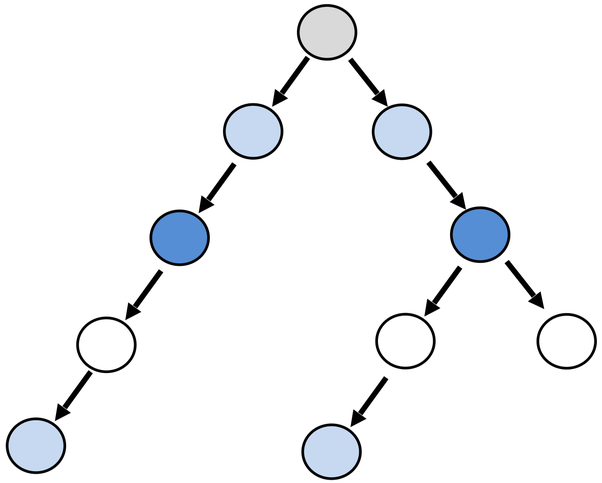
Robot Motion & Planning

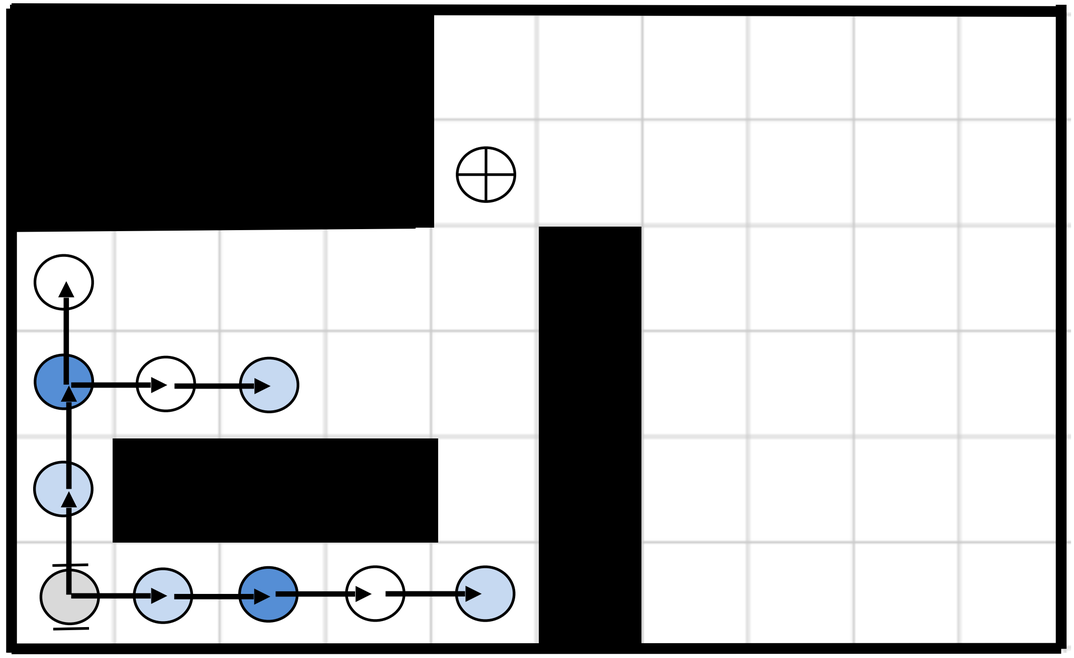




ROB-UY 3303

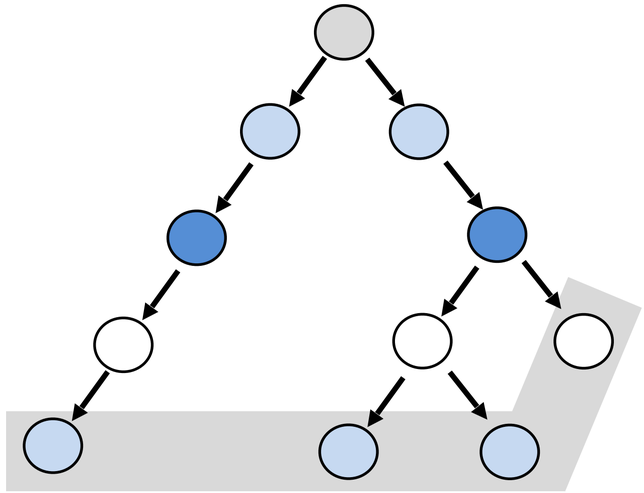
Robot Motion & Planning

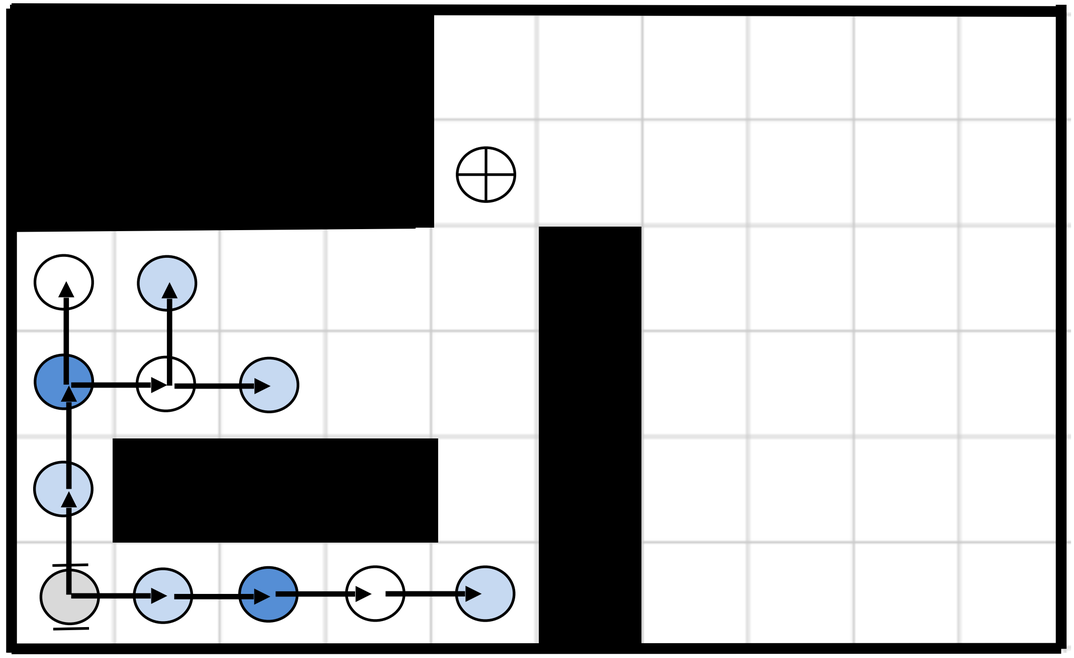


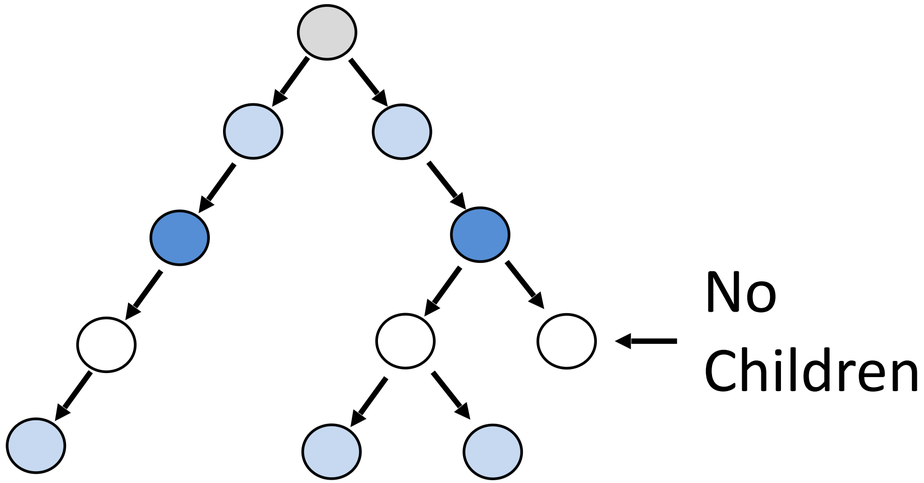


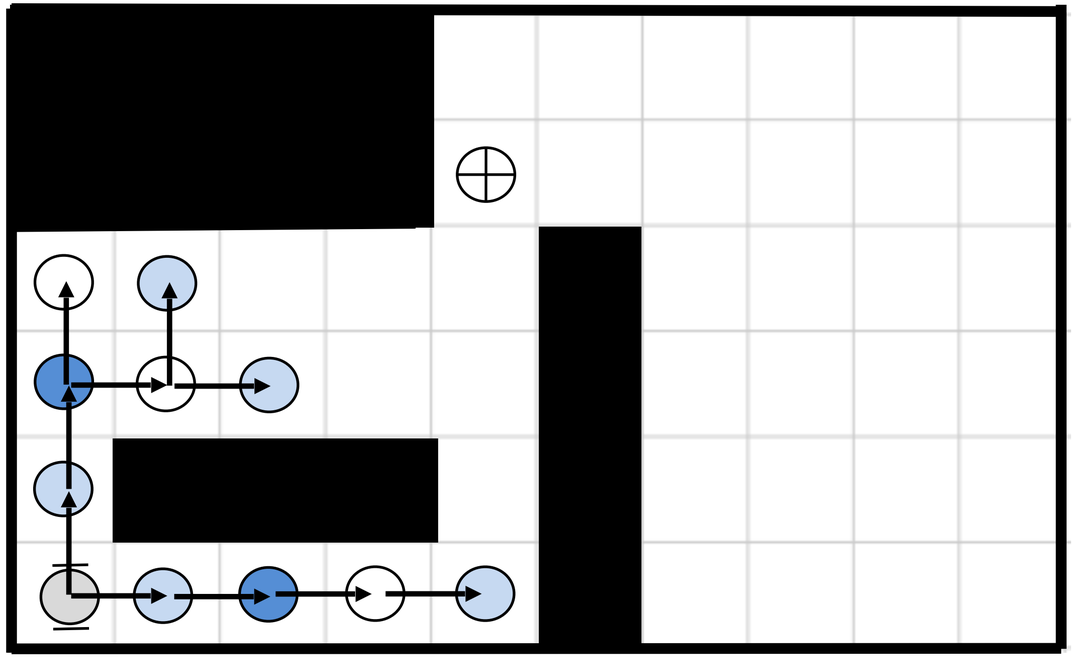
ROB-UY 3303

Robot Motion & Planning



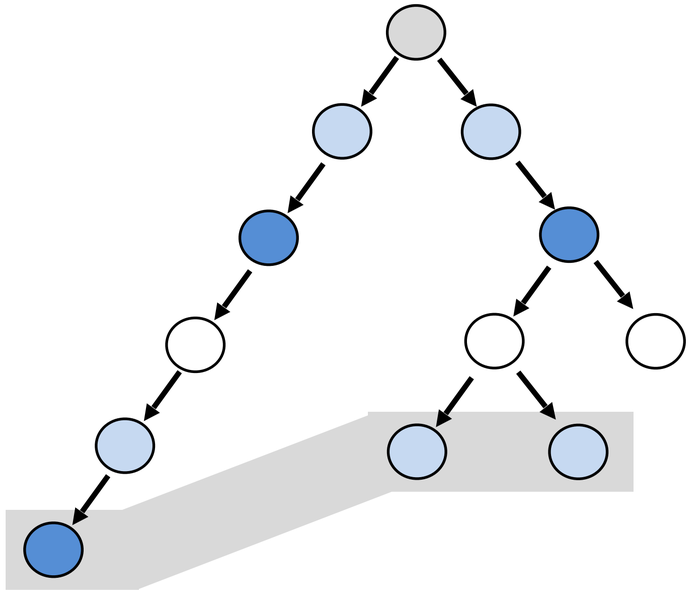


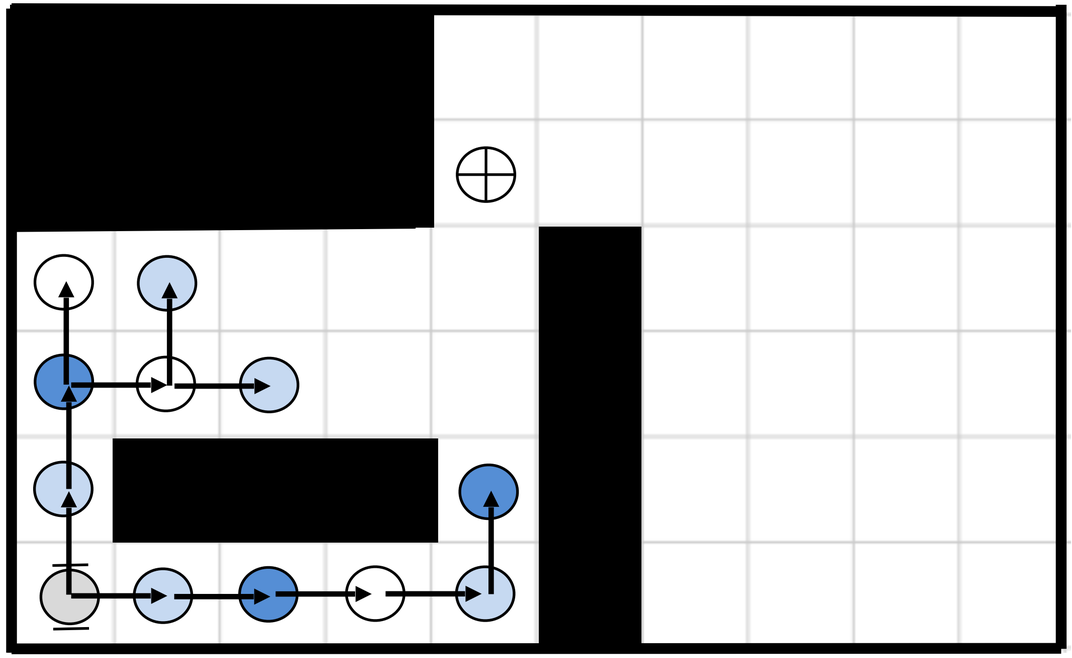

No
Children



ROB-UY 3303

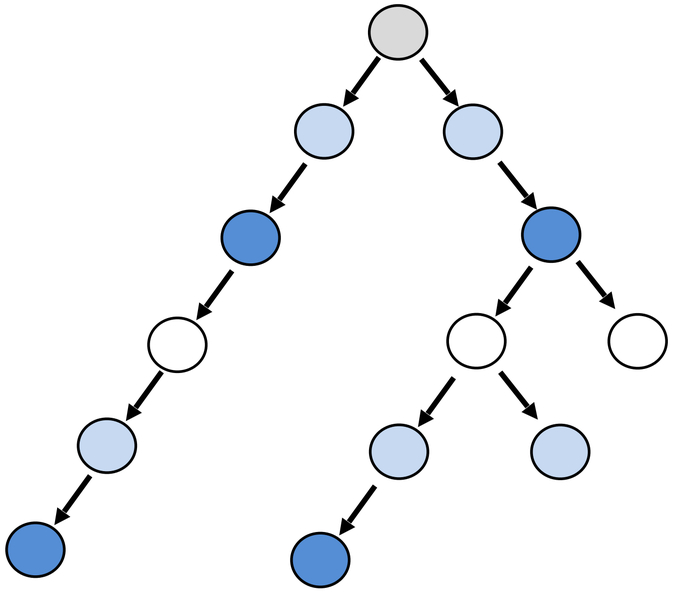
Robot Motion & Planning

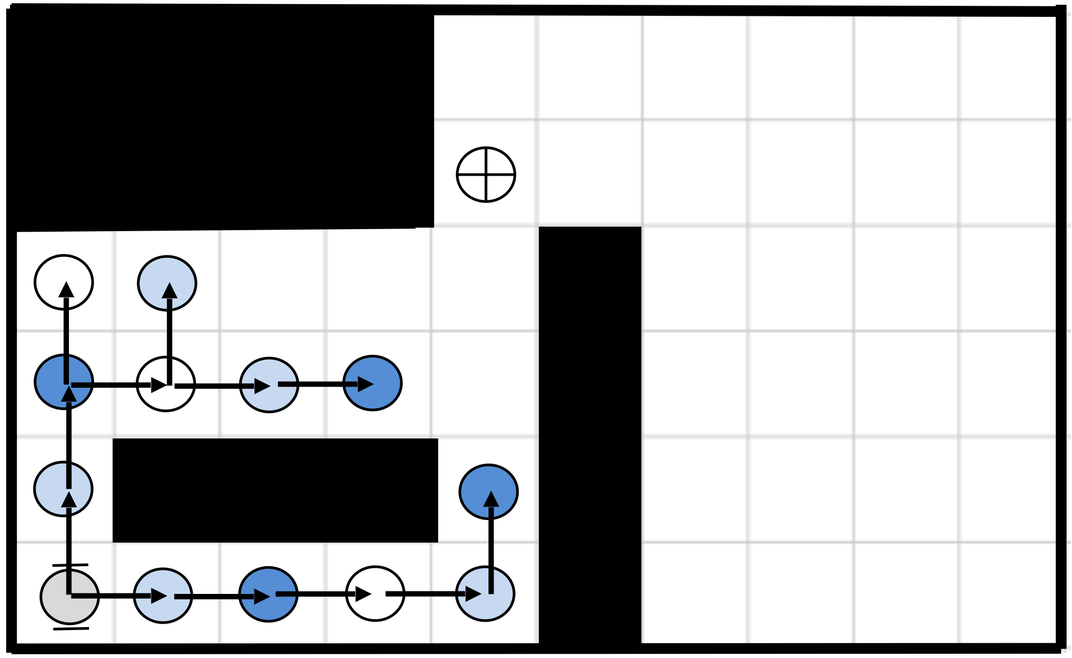




ROB-UY 3303

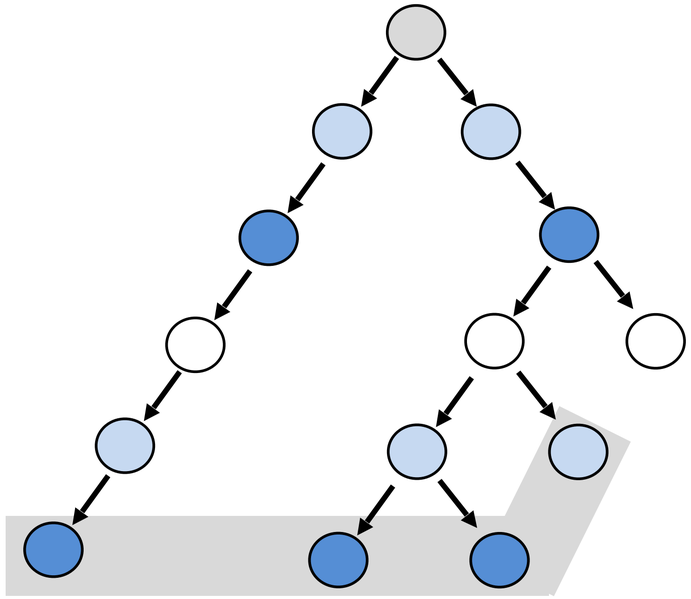
Robot Motion & Planning

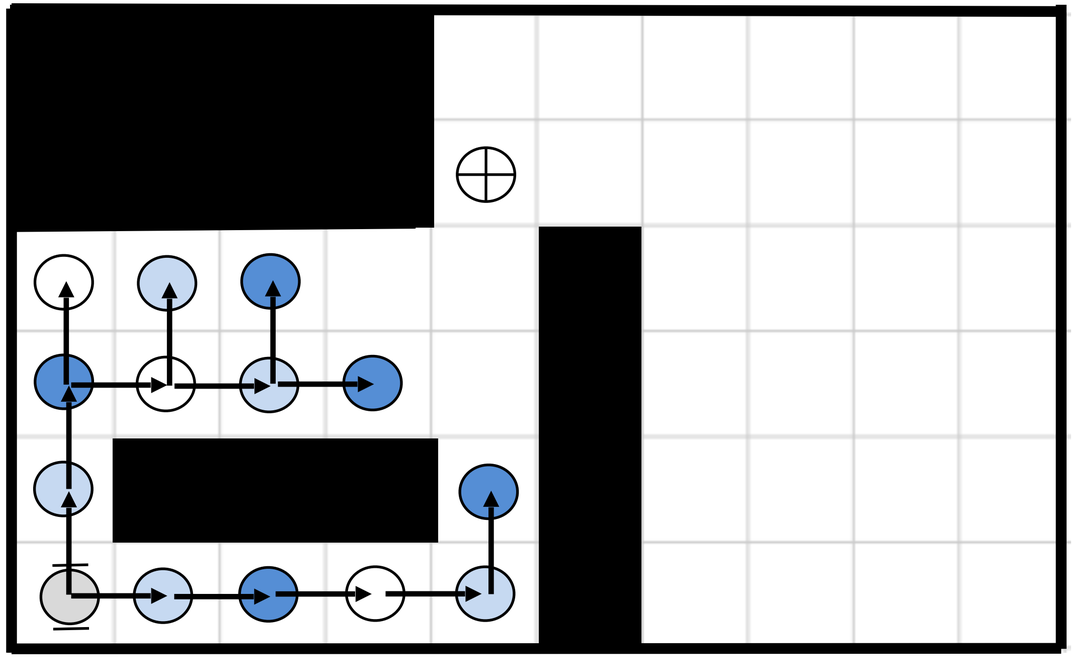


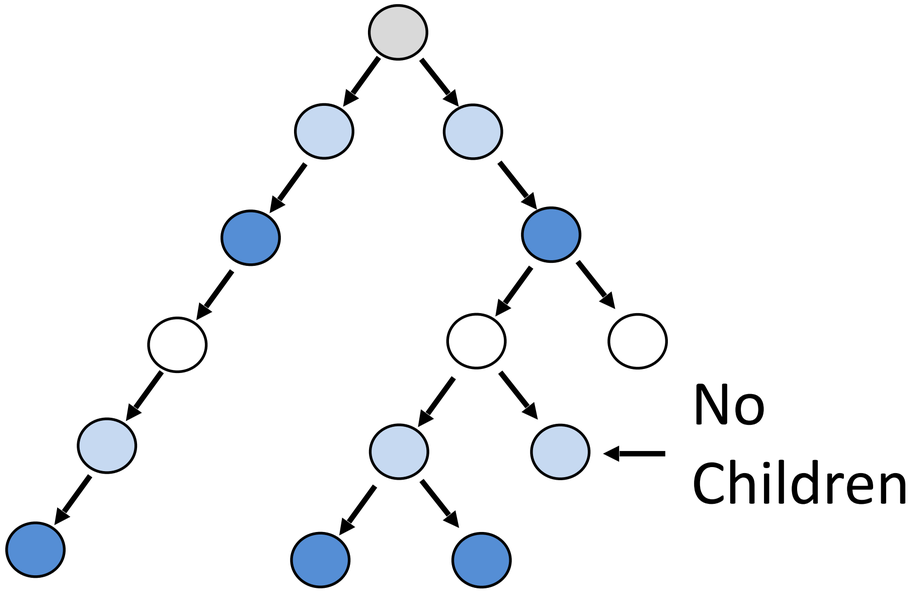


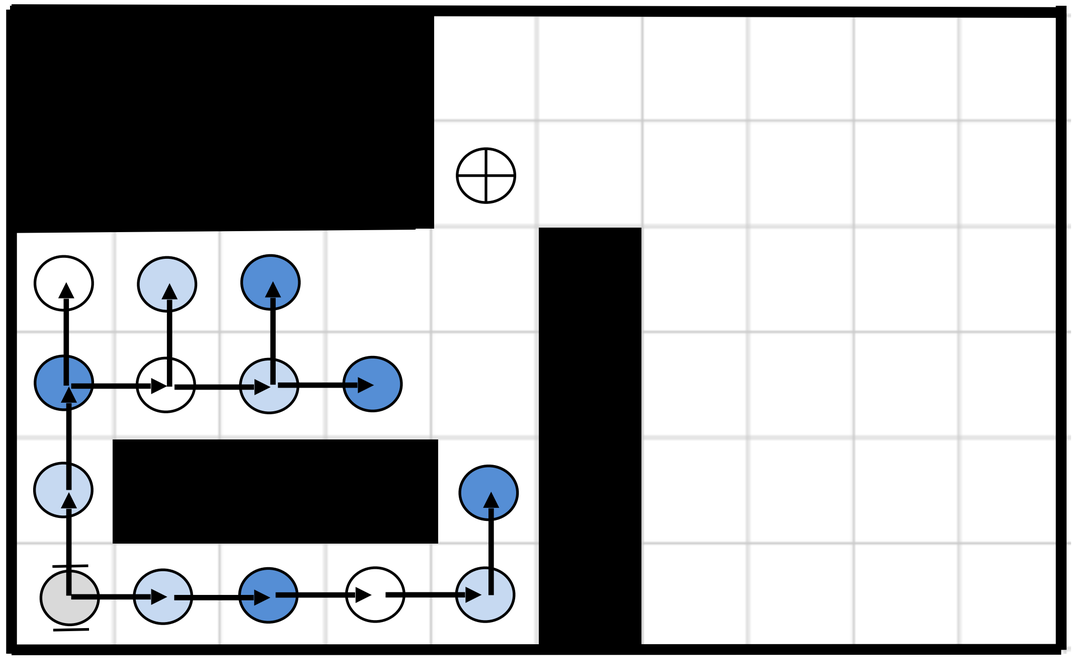
ROB-UY 3303

Robot Motion & Planning



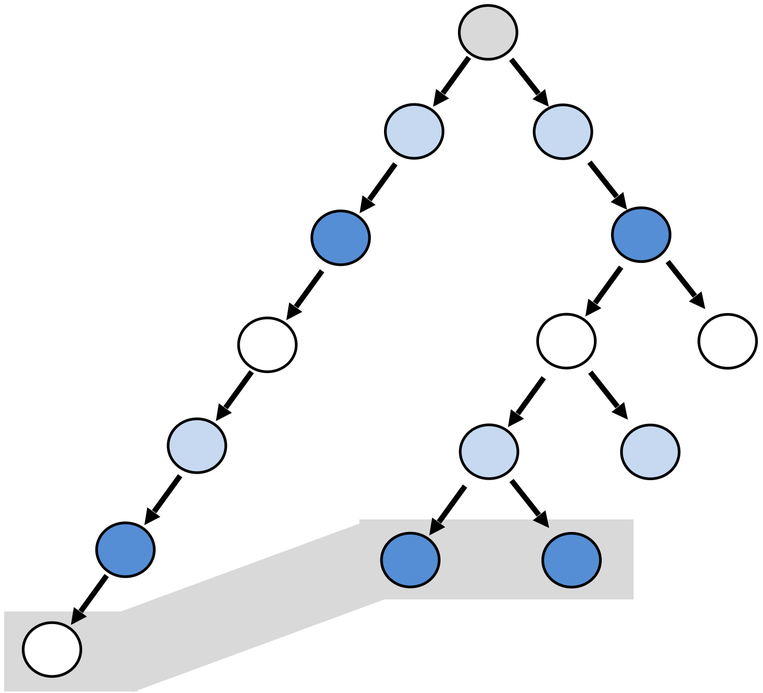


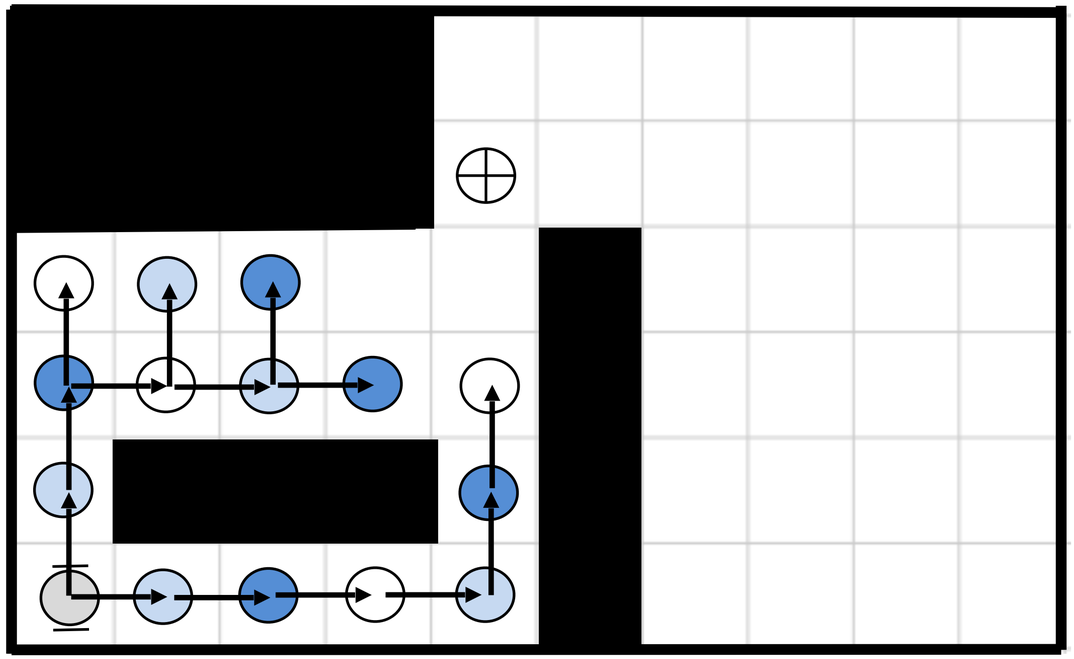

No
Children



ROB-UY 3303

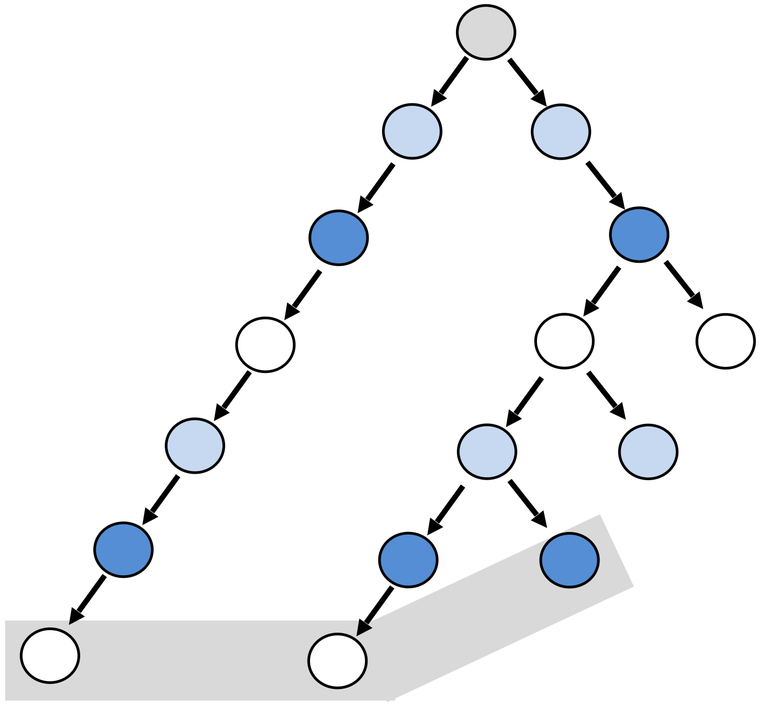
Robot Motion & Planning

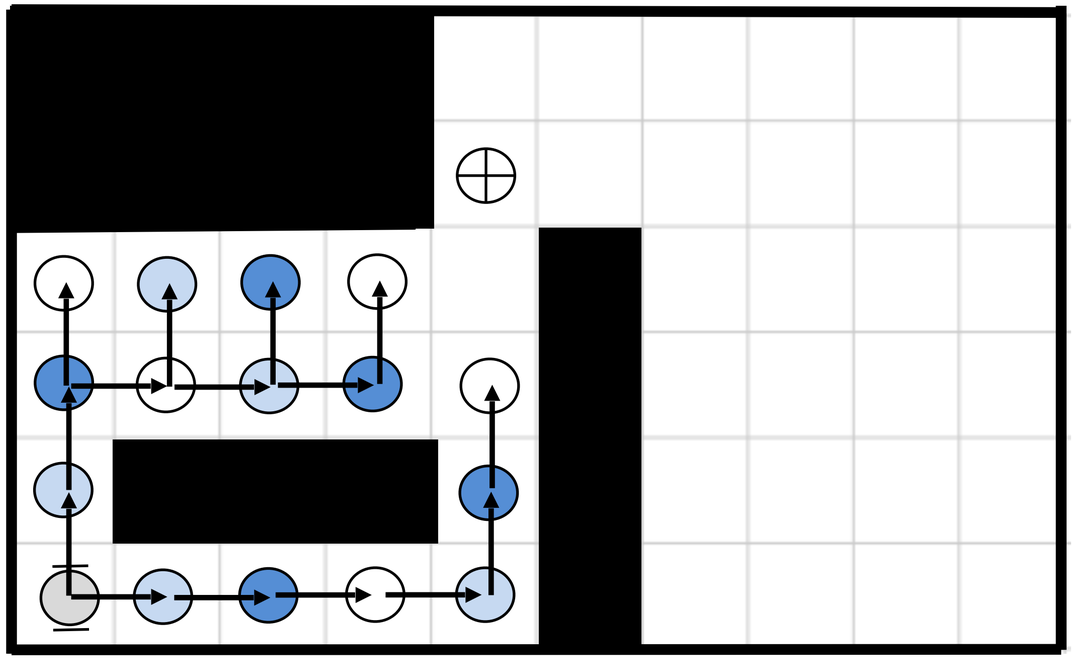


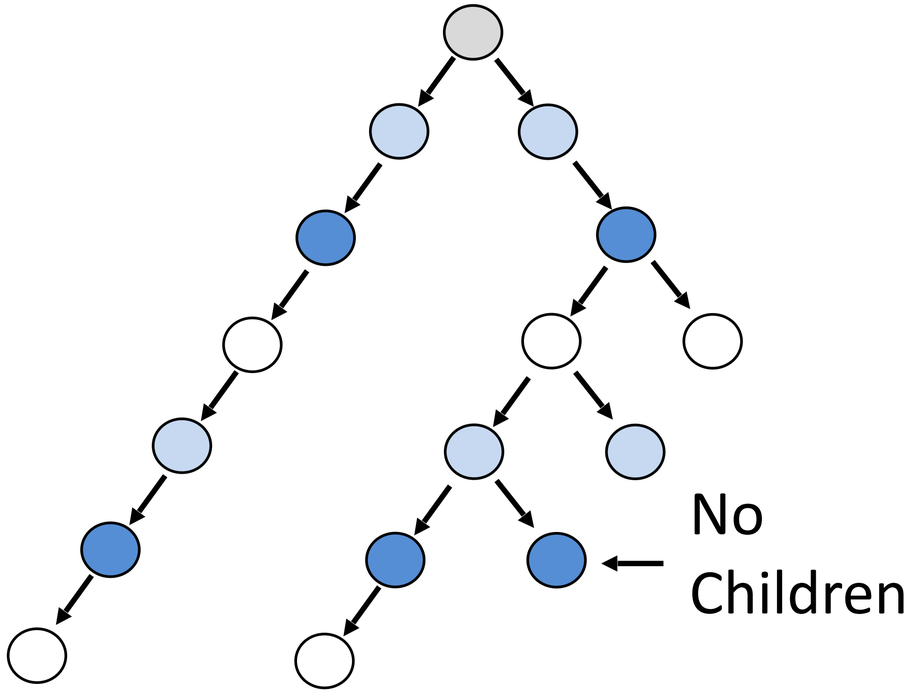


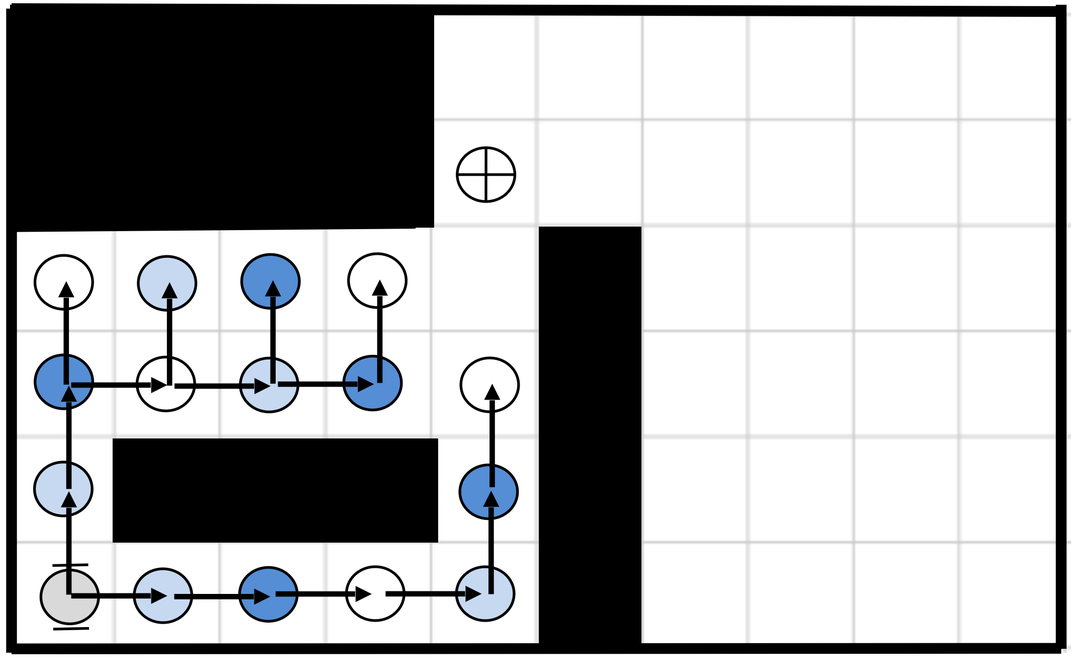
ROB-UY 3303

Robot Motion & Planning



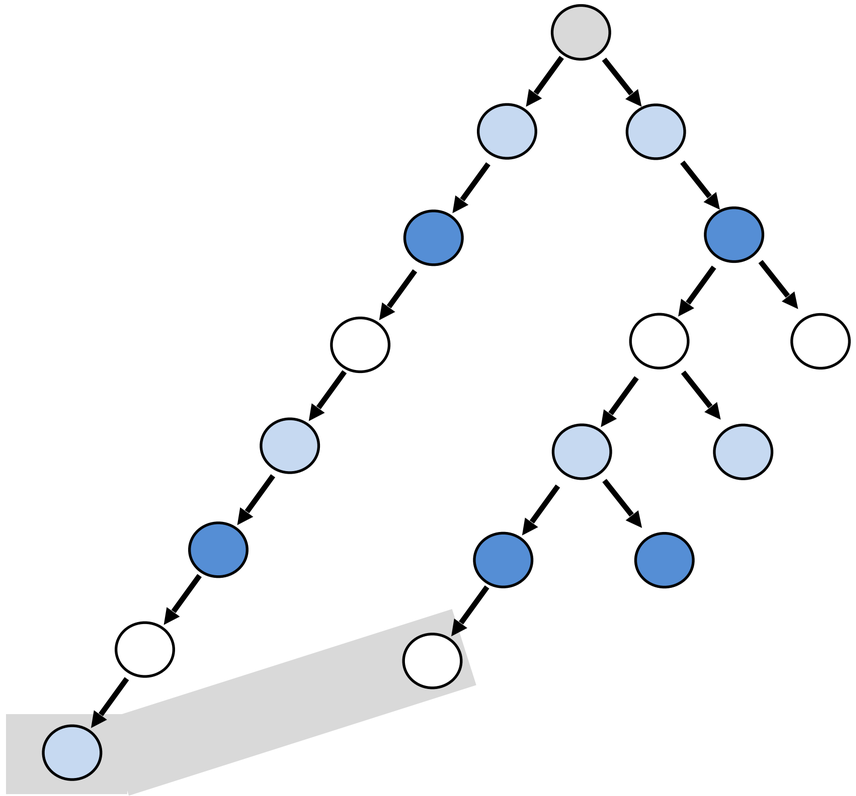


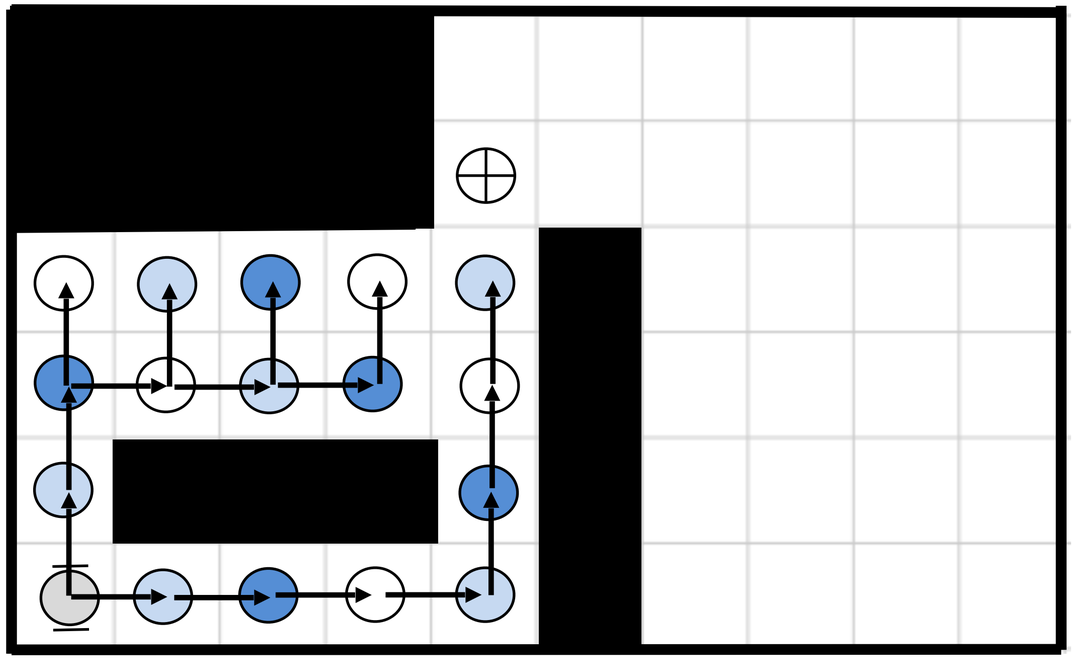

No
Children

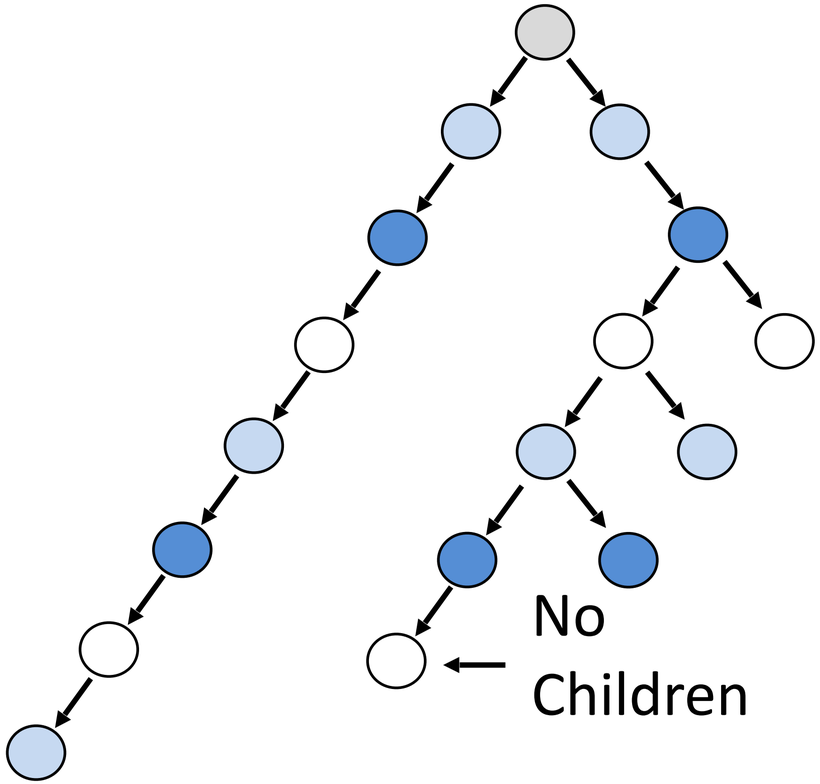


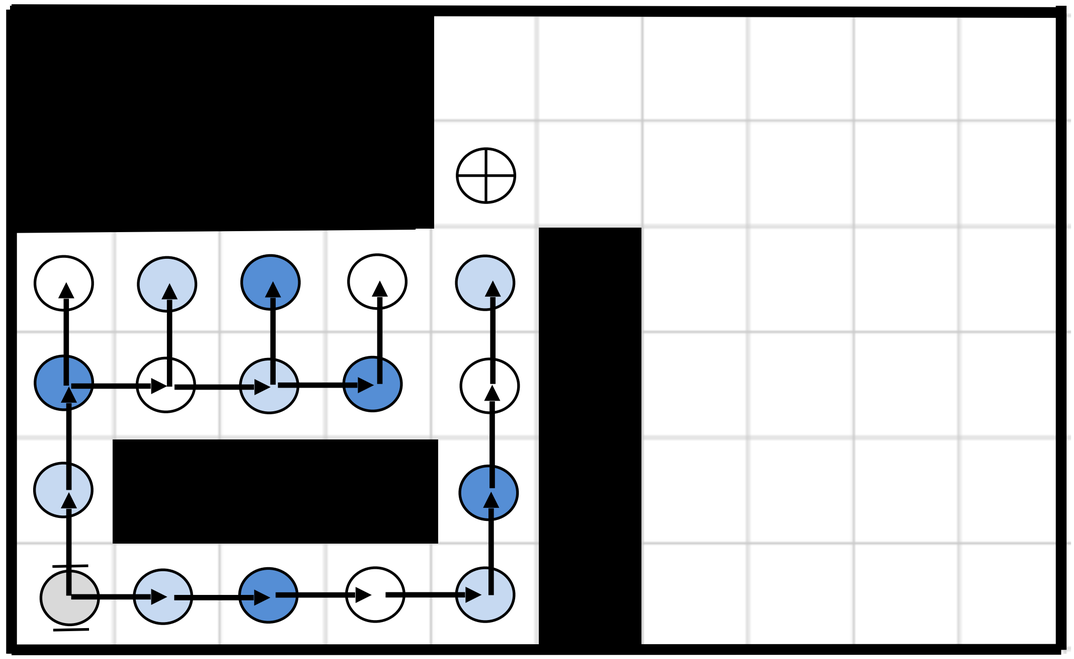
ROB-UY 3303

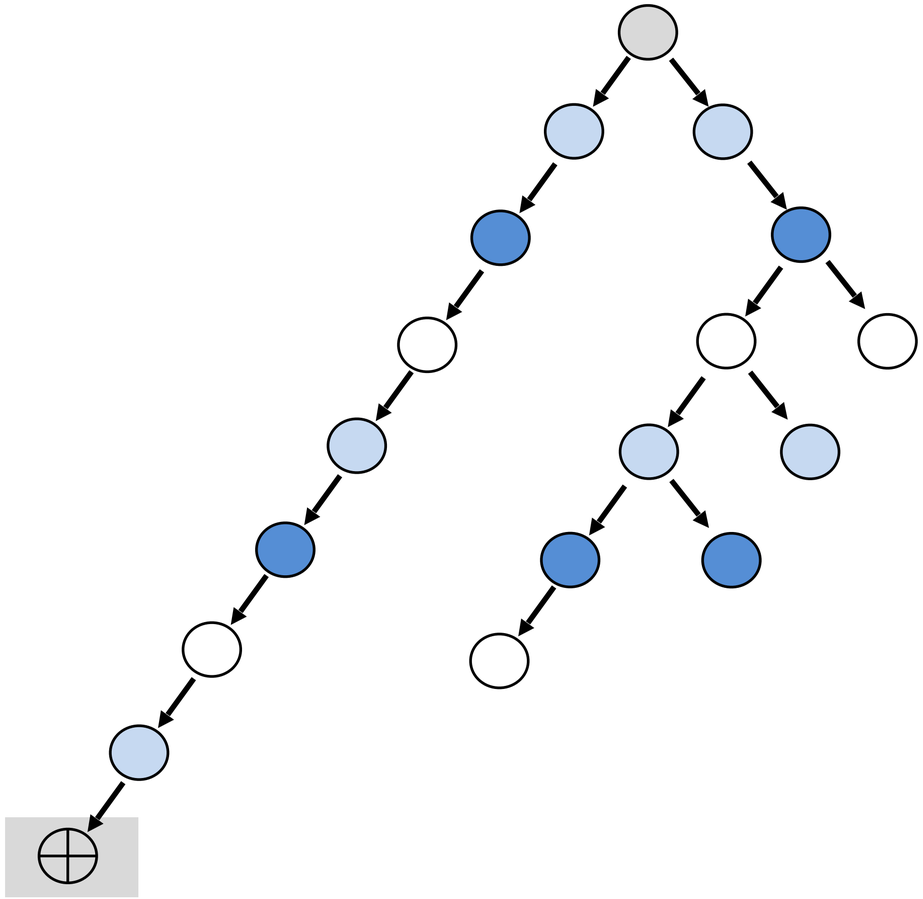
Robot Motion & Planning

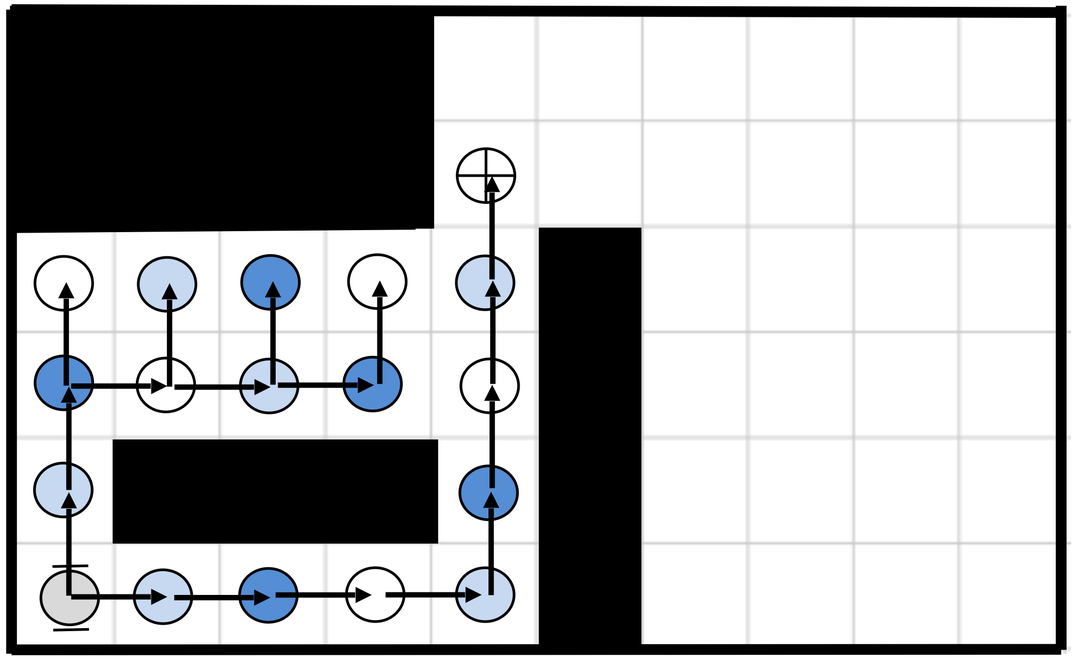


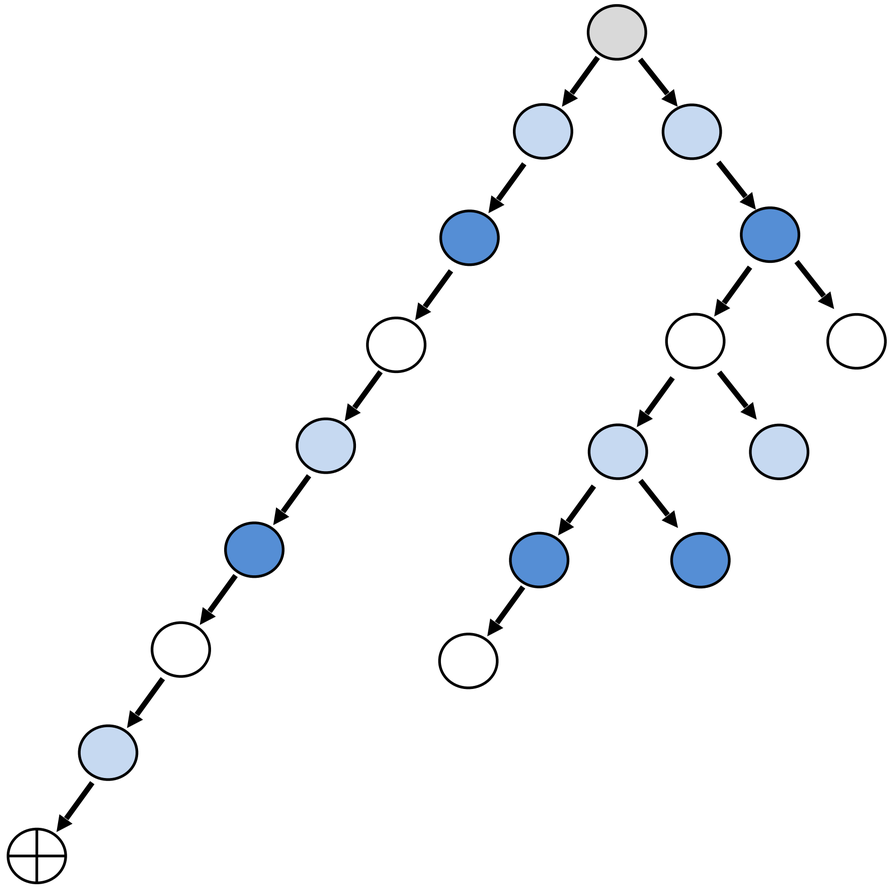


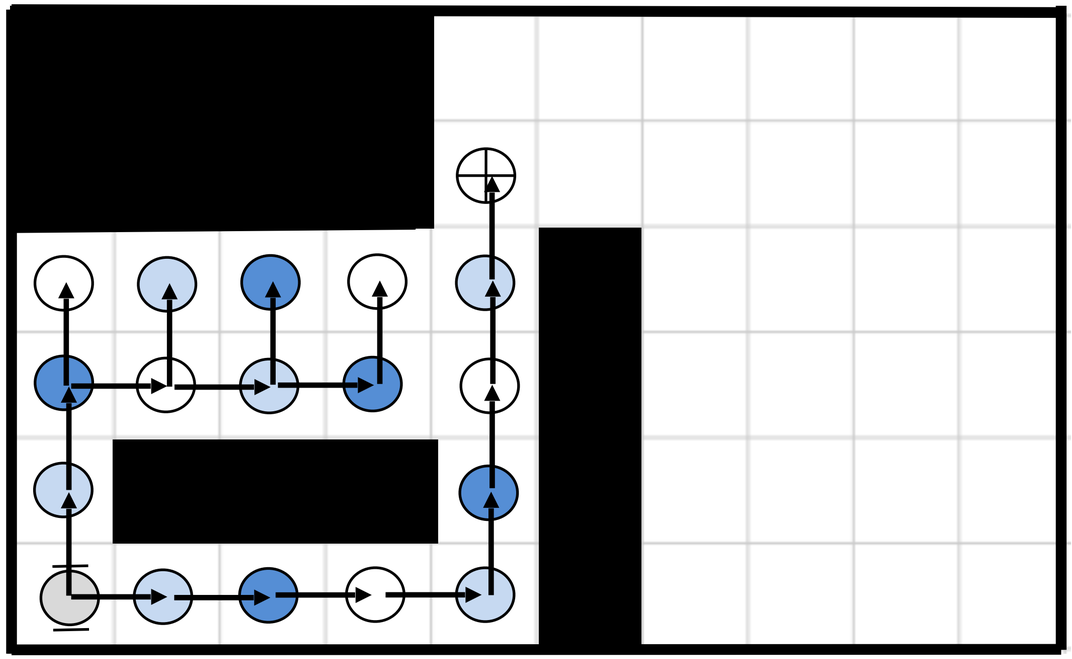

No
Children

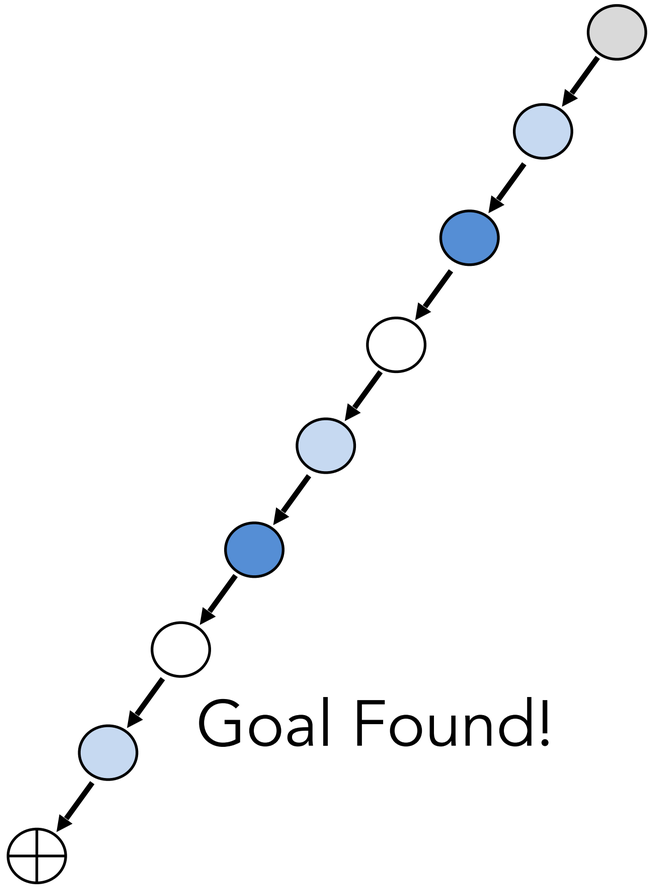


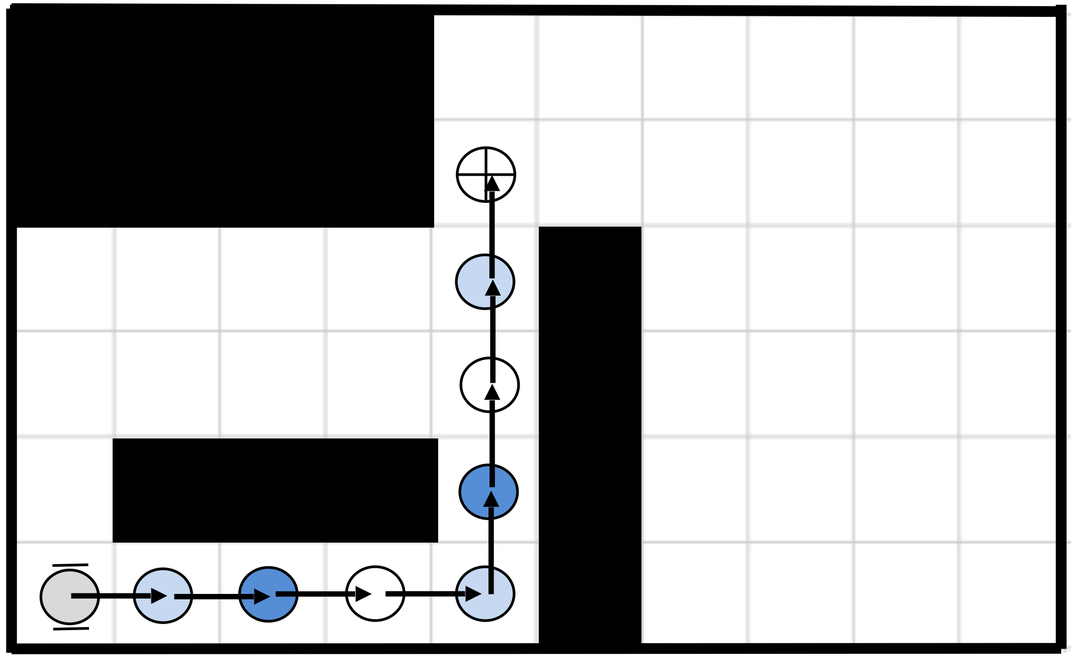






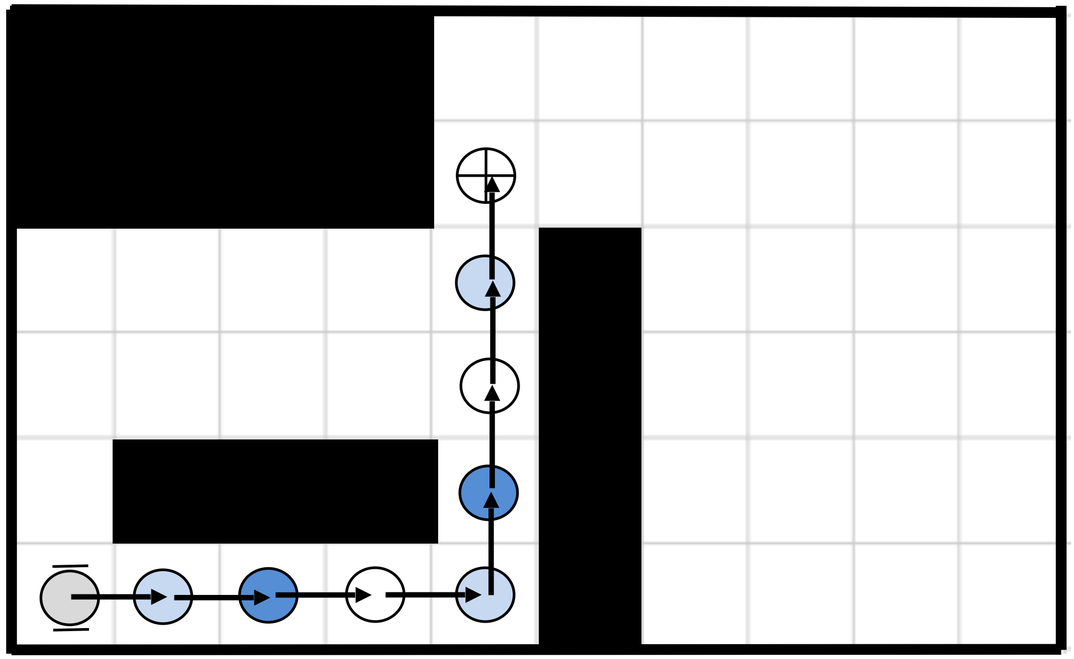



Goal Found!



- **Characteristics**

  - The algorithm is complete since it systematically checks all possible options.
  - If path cost is a uniform and non-decreasing function of the depth, then the algorithm can be optimal (e.g. path cost = 1 for each action)



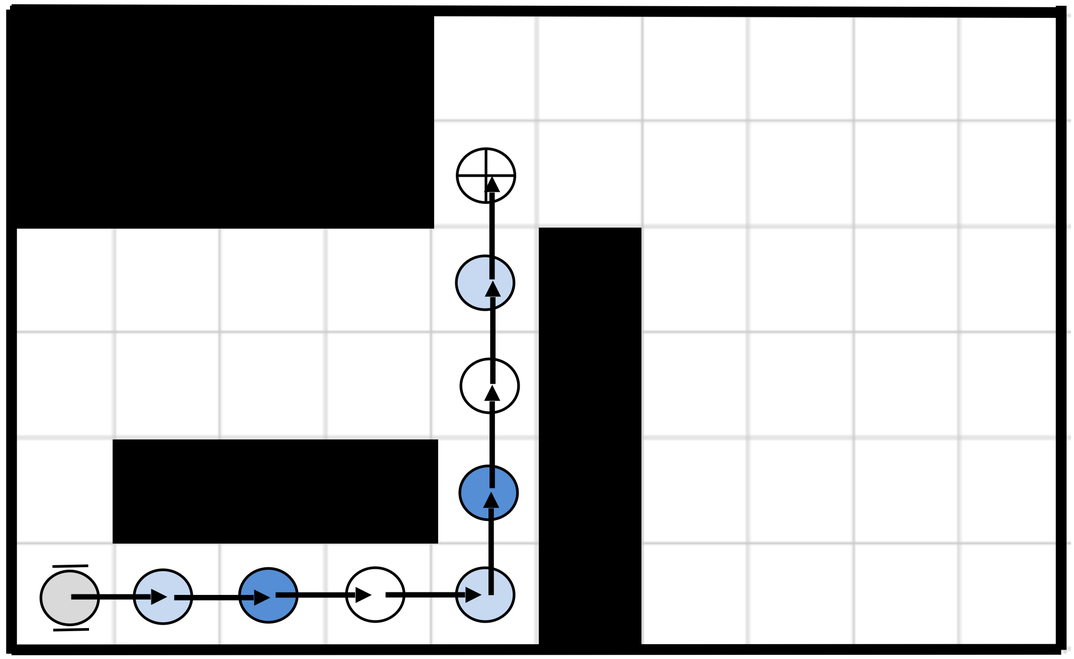
- To determine **time complexity**, assume that each node has b successors (we call b the **branching factor**)

- Similarly, the set of expansions at depth d has b^d nodes generated.
- If the solution is found at depth d, then the total number of goal tests is

$$1 + b^1 + b^2 + b^3 + \ldots + b^d$$

- Hence the **time complexity** is $O(b^d)$

- **BFS Characteristics**
  - Speed
    - $O(b^d)$ run time complexity
  - Completeness
    - Yes
  - Optimality
    - Yes (wrt resolution)
  - Feasibility
    - Not guaranteed

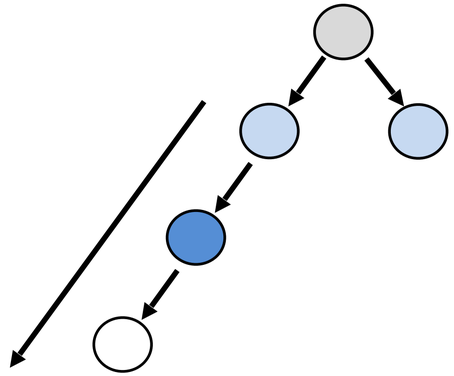


Agenda

1. The Graph
2. Graph Search
3. Breadth First Search
4. **Depth First Search**

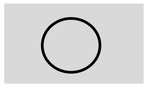
Depth First Search

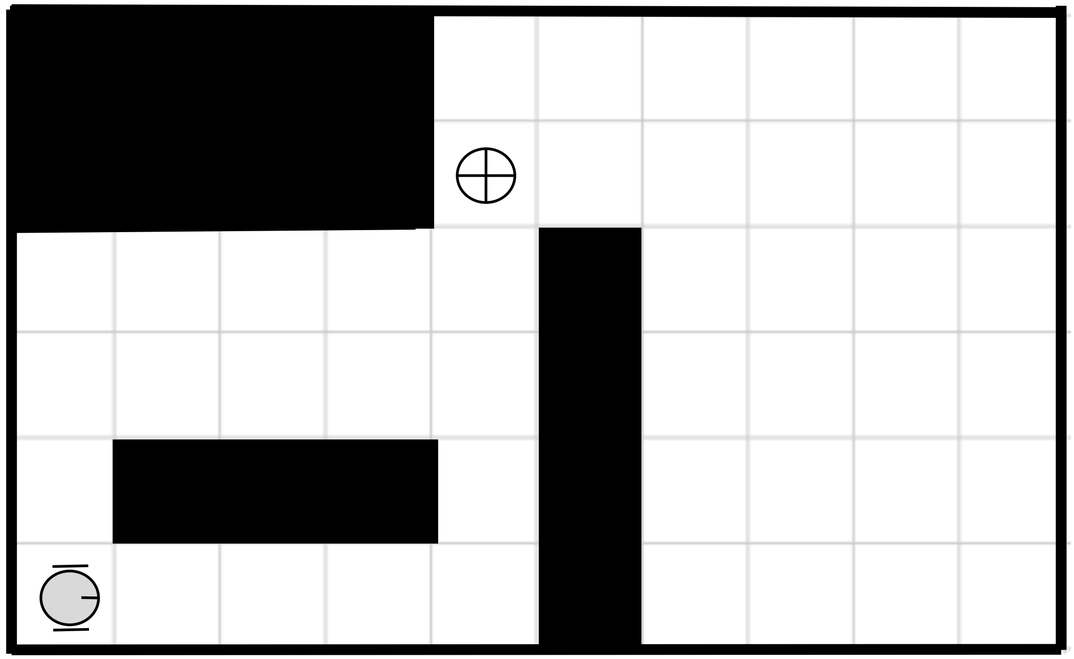
- Uses a strategy where nodes are expanded as deep as possible
  - i.e. All nodes of depth level d will be expanded before any node of depth level < d



ROB-UY 3303

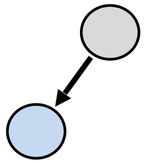
Robot Motion & Planning

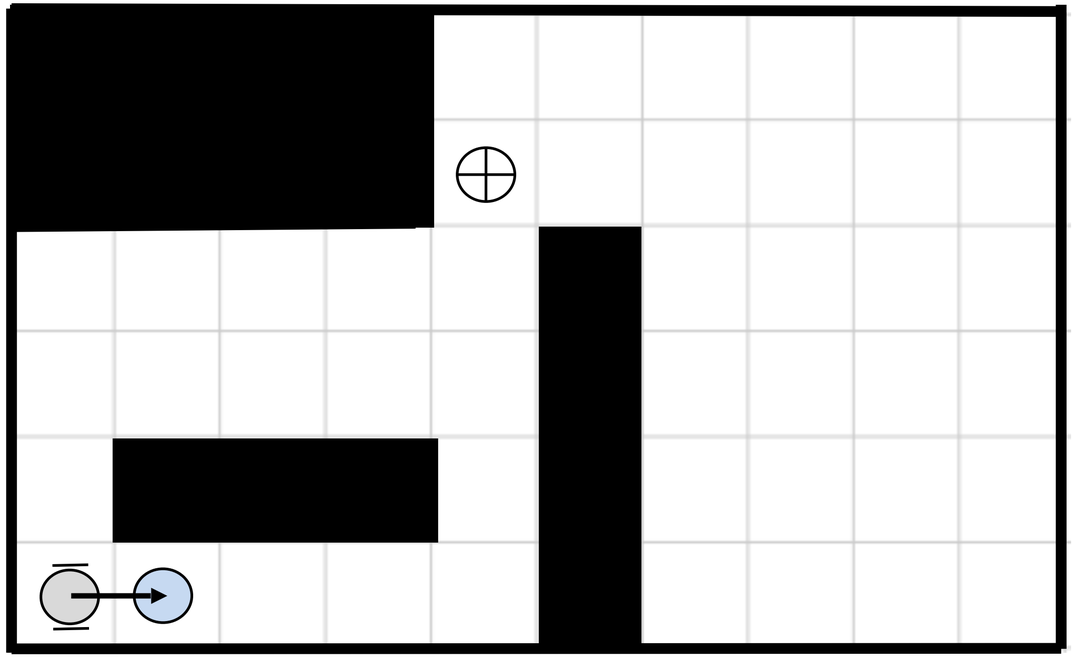




ROB-UY 3303

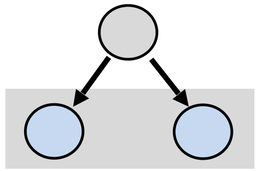
Robot Motion & Planning

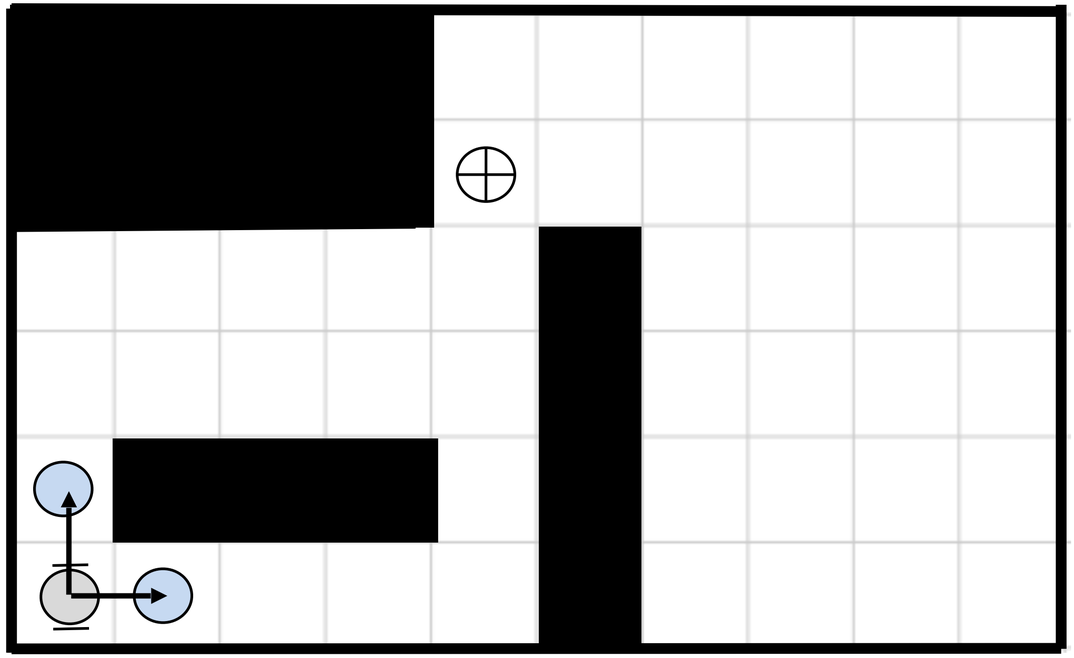




ROB-UY 3303

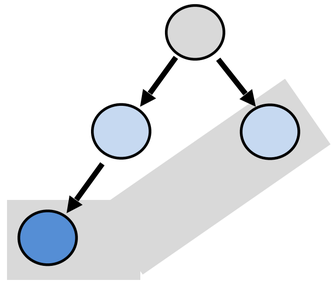
Robot Motion & Planning

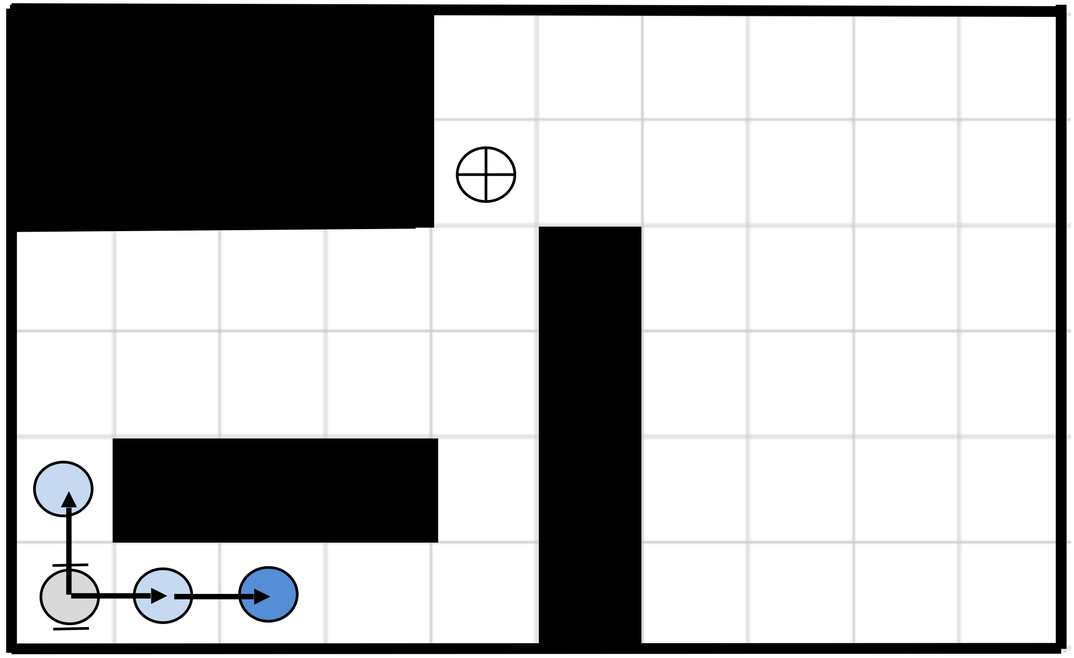




ROB-UY 3303

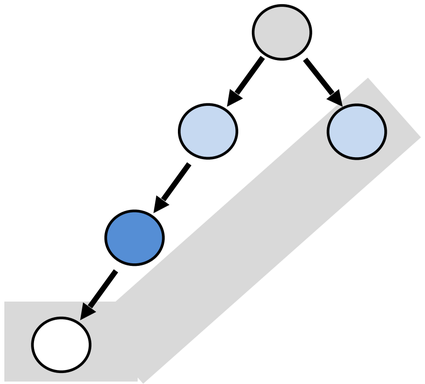
Robot Motion & Planning

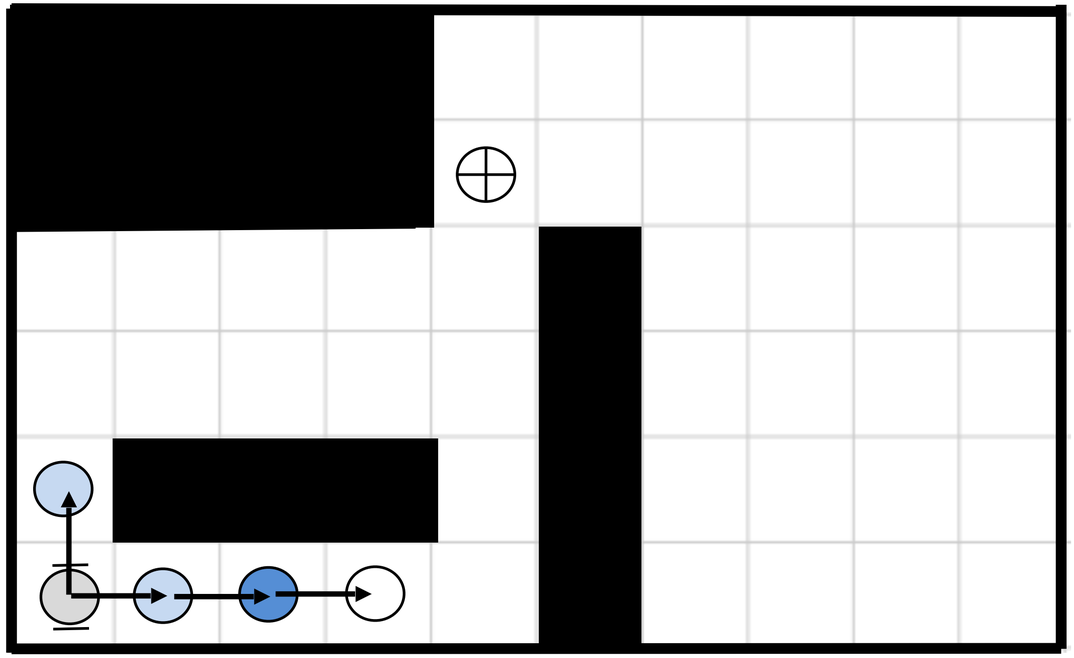




ROB-UY 3303

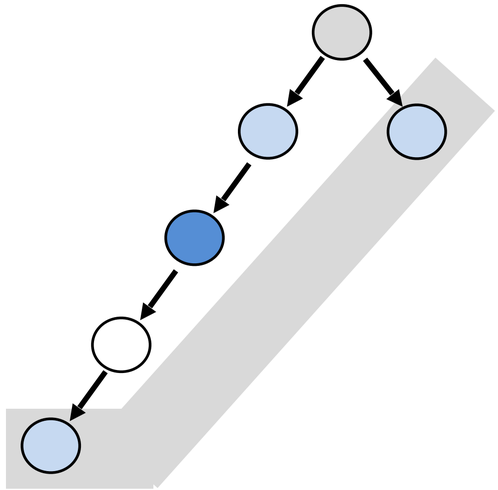
Robot Motion & Planning

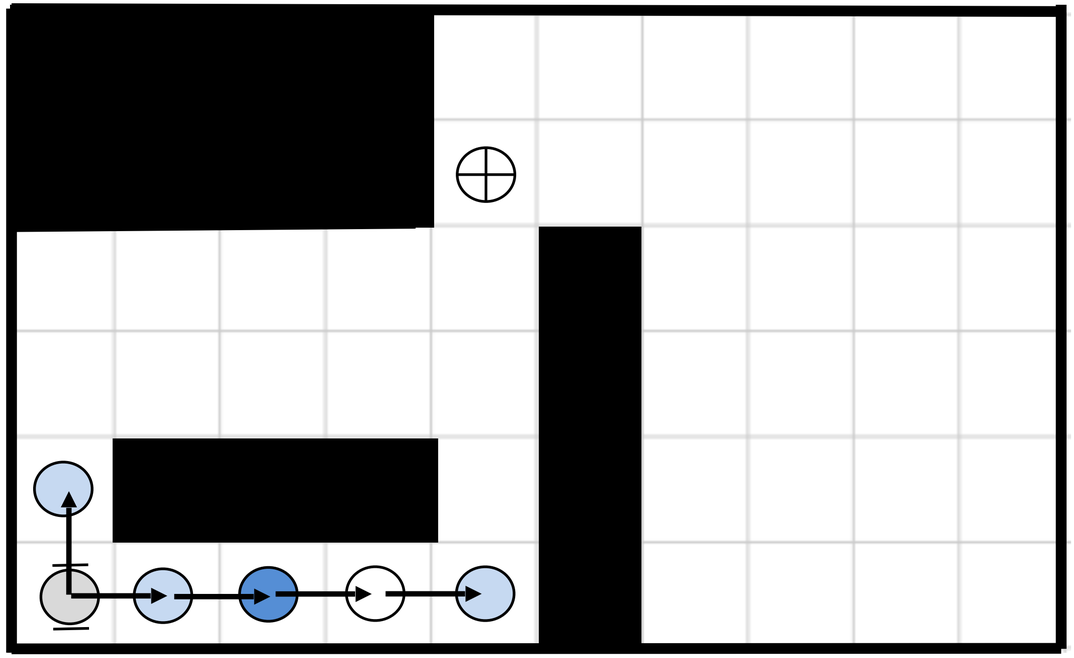




ROB-UY 3303

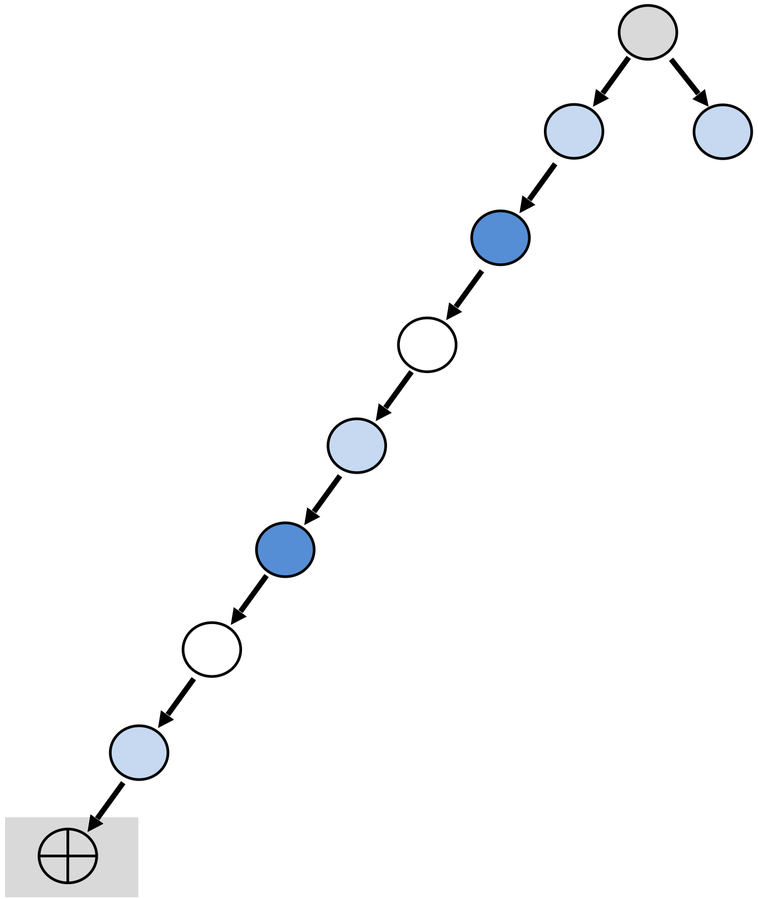
Robot Motion & Planning

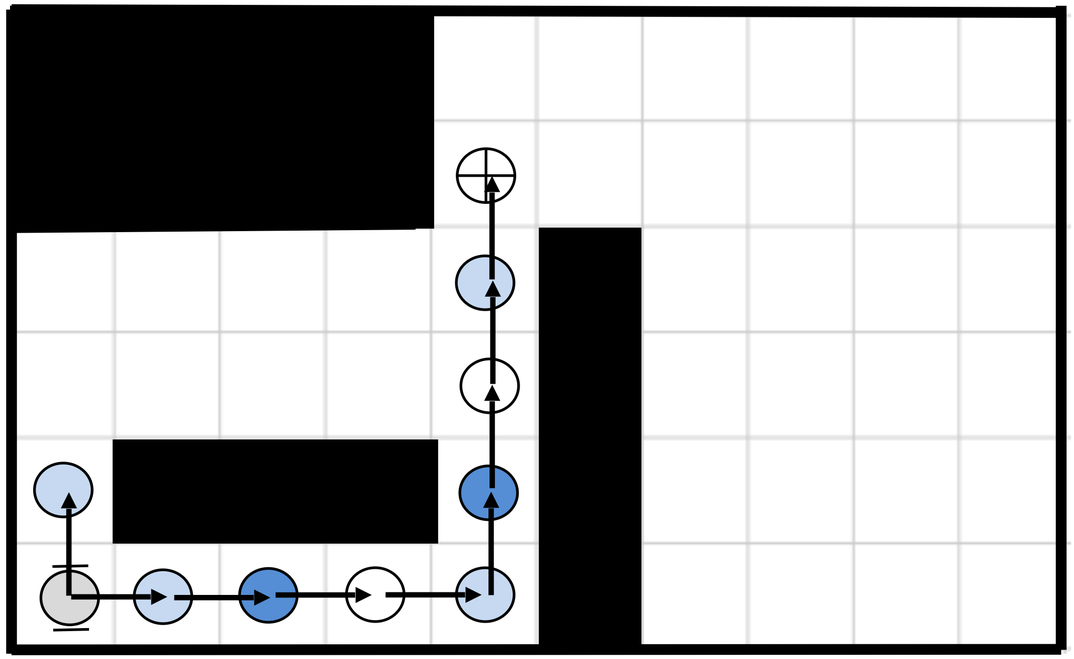


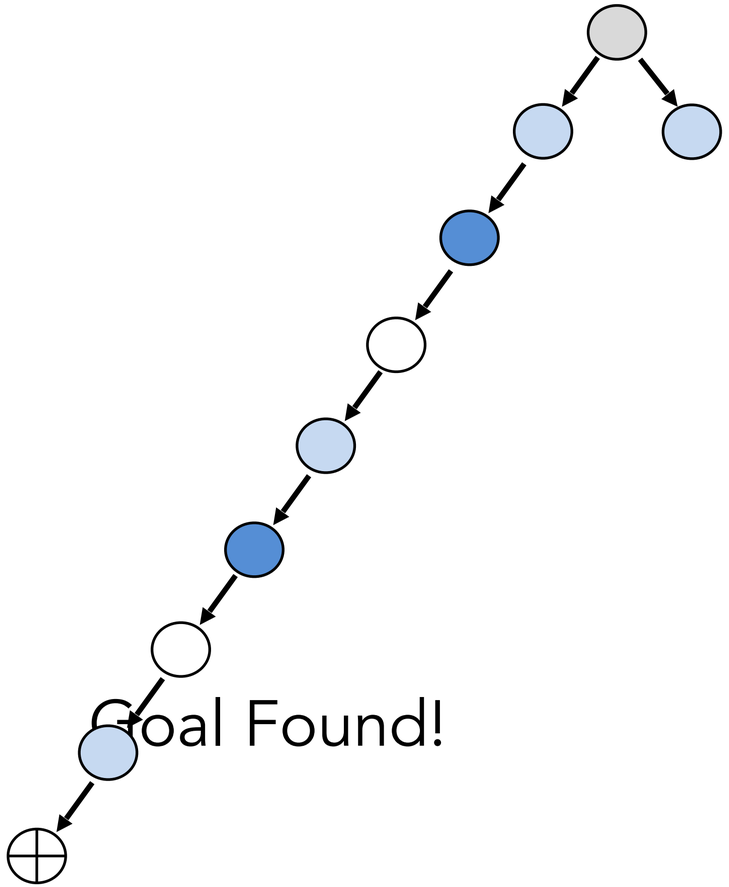


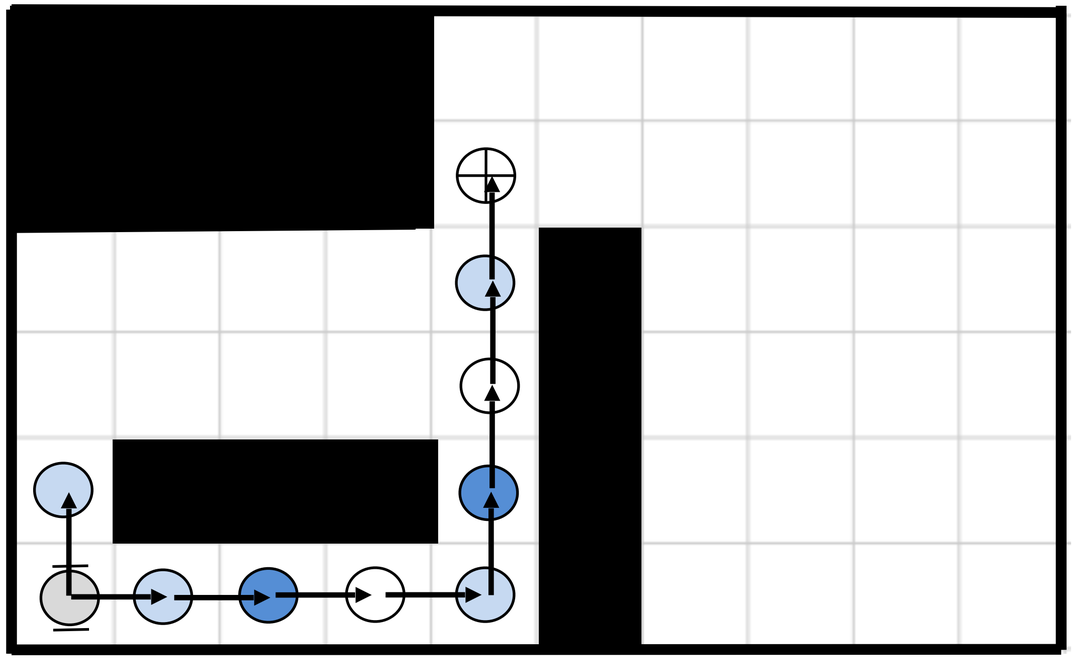
ROB-UY 3303

Robot Motion & Planning



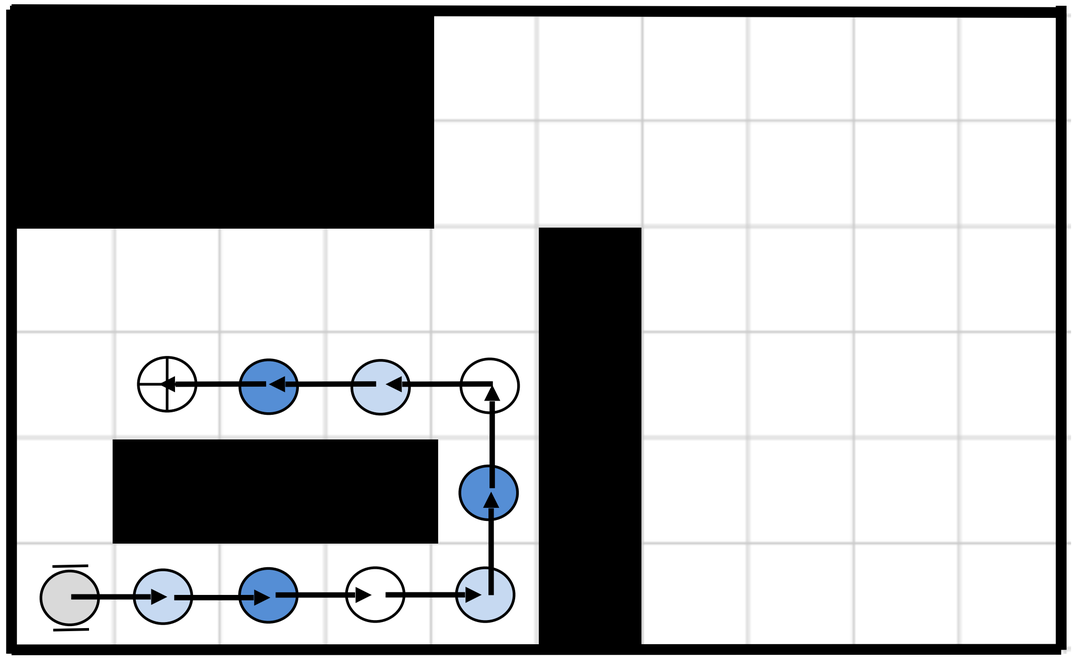



Goal Found!

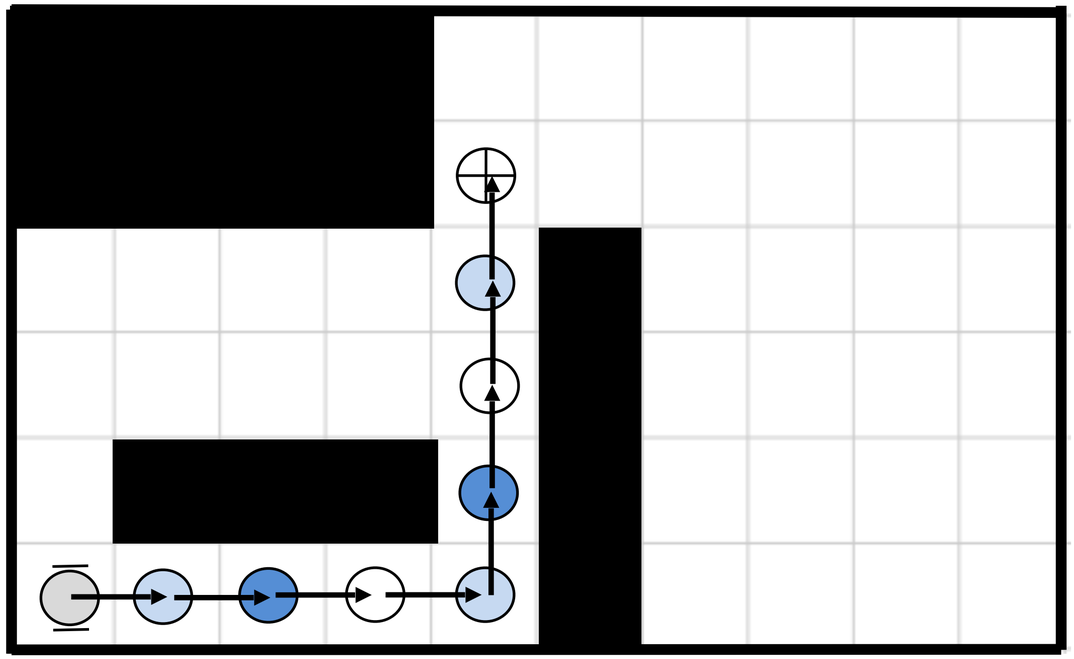


DFS Characteristics

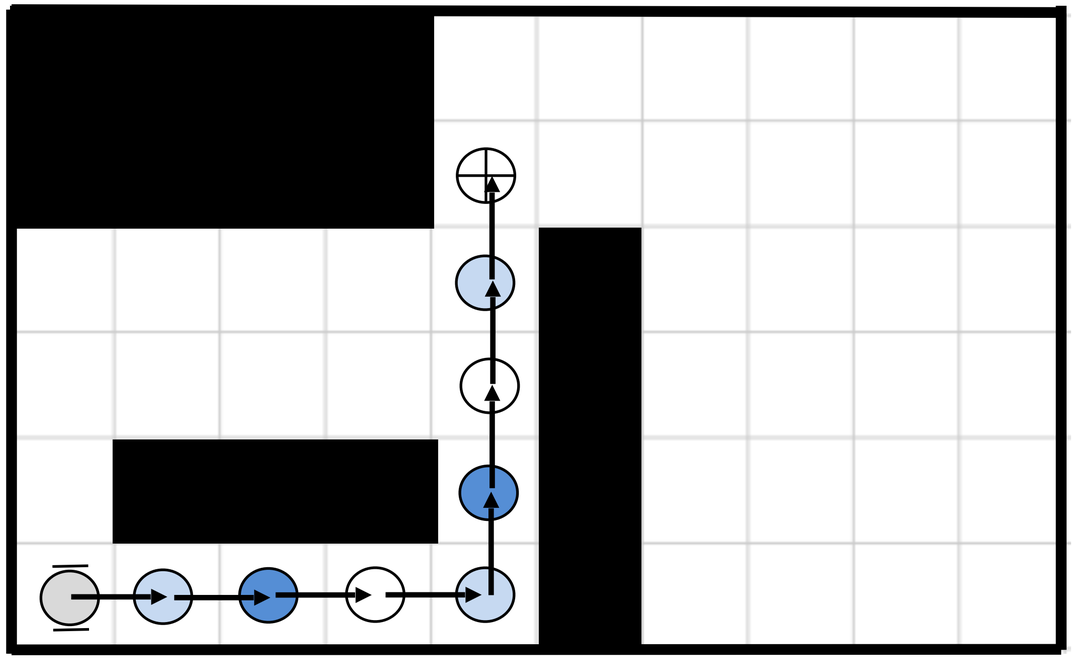
- The algorithm is complete since it systematically checks all possible options.
  - Assuming no cycles!
- If we return the first solution found, we may not find the optimal solution!



- **DFS Characteristics**
  - Speed
    - $O(b^d)$ run time complexity
  - Completeness
    - Yes
  - Optimality
    - Yes (wrt resolution)
  - Feasibility
    - Not guaranteed



- **Application dependent**
  - BFS can be optimal
    - One solution of minimum value -> BFS
  - DFS better on memory
    - Many long solutions of equal value -> DFS



- **Summary**
  - Different search algorithms differ by how they select the next node to expand.
  - Choose algorithm based on application

